

BỘ GIÁO DỤC ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM VÀ KỸ THUẬT TP. HCM
KHOA CHẤT LƯỢNG CAO
○○○○



HCMUTE

BÁO CÁO CUỐI KỲ

Môn học: Cơ Sở Dữ Liệu Nâng Cao

Đề tài:

**XÂY DỰNG HỆ THỐNG
QUẢN LÝ THƯ VIỆN**

Mã môn học:

Giảng viên hướng dẫn: ThS. LÊ THỊ MINH CHÂU

Sinh viên thực hiện: Cao Thị Ngọc Phụng 21110276

Trần Văn Tiến 21110319

Đinh Thị Thúy Quỳnh 21110284

Tp. Hồ Chí Minh, tháng 6 năm 2025

ĐH SƯ PHẠM KỸ THUẬT TP.HCM

CỘNG HÒA XHCN VIỆT NAM

PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN**Tên đề tài:** Xây dựng hệ thống quản lý thư viện**Danh sách sinh viên tham gia**

STT	Họ và tên	Mã số sinh viên	Tỷ lệ tham gia
1	Cao Thị Ngọc Phụng	21110276	100%
2	Trần Văn Tiến	21110319	100%
3	Đinh Thị Thúy Quỳnh	21110284	100%

*Ghi chú: Tỷ lệ %: Mức độ phần trăm hoàn thành của từng sinh viên tham gia.***Thời gian làm đồ án:** Từ ngày 26/04/2025 đến ngày 25/5/2025**Giảng viên hướng dẫn:** ThS. Lê Thị Minh Châu**Trưởng nhóm:** Cao Thị Ngọc Phụng**Điểm số:****Nhận xét của giáo viên:**

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh - Tháng 5 năm 2025

BẢNG KẾ HOẠCH LÀM VIỆC VÀ PHÂN CÔNG NHIỆM VỤ

Cao Thị Ngọc Phụng	- Tìm hiểu về MongoDB, .Net - Phân tích và thiết kế csdl về Account, Staff, Reader - Tìm hiểu và kết nối MongoDB với Winform - Thiết kế giao diện liên quan đến Đăng nhập, Tài khoản, Quản lý thủ thư, Quản lý người đọc, Thống kê số liệu - Code back-end về các file liên quan đến Account, Staff, Reader (Find, Insert, Update, Delete, Aggregate) [Mục 3.4.1, Mục 3.4.2, Mục 3.4.3, Mục 3.4.4, Mục 3.4.5 , Mục 3.4.6, Mục 3.4.12] - Tìm hiểu và tổng hợp báo cáo chương 3
Trần Văn Tiến	- Tìm hiểu về MongoDB, .Net - Phân tích và thiết kế csdl về Loan, Transaction - Thiết kế giao diện liên quan đến Quản lý giao dịch, Quản lý phí giao dịch - Code back-end về các file liên quan đến Loan, Transaction (Find, Insert, Update, Delete) [Mục 3.4.10, Mục 3.4.11] - Tìm hiểu và tổng hợp báo cáo chương 1
Đinh thị Thúy Quỳnh	- Tìm hiểu về MongoDB, .Net - Phân tích và thiết kế csdl về Author, Category, Book - Code back-end về các file liên quan đến Author, Category, Book (Find, Insert, Update, Delete) - Thiết kế giao diện liên quan đến Quản lý tác giả, Quản lý thể loại, Quản lý sách [Mục 3.4.7, Mục 3.4.8, Mục 3.4.9] - Tìm hiểu và tổng hợp báo cáo chương 2

MỤC LỤC

PHẦN MỞ ĐẦU	1
1. Tính cấp thiết của đề tài.....	1
2. Mục tiêu của đề tài.....	1
3. Phạm vi nghiên cứu.....	1
PHẦN NỘI DUNG.....	3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG	3
1.1 Hệ thống thư viện.....	3
1.1.1 Tổng quan.....	3
1.1.2 Thực trạng	3
1.1.2.1 Ưu điểm	4
1.1.2.1 Hạn chế.....	4
1.2 Các công nghệ sử dụng.....	5
1.2.1 MongoDB.....	5
1.2.1.1 Giới thiệu về MongoDB	5
1.2.1.2 MongoAtlas	6
1.2.1.3 Công cụ truy vấn dữ liệu	6
1.2.1.4 Dữ liệu phi cấu trúc	6
1.2.1.5 So sánh dữ liệu có cấu trúc và phi cấu trúc	7
1.2.2 WinForms (.NET)	8
1.2.2.1 Giới thiệu về Winform (.NET)	8
1.2.2.2 C# Programming.....	8
1.2.3 Kiến trúc ứng dụng.....	8
1.2.3.1 Mô hình 3 lớp (Three-Tier Architecture)	8
1.2.3.2 Mô hình Client–Server	9
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	10
2.1 Phân tích yêu cầu hệ thống.....	10
2.1.1 Yêu cầu chức năng	10
2.1.1.1 Quản lý tài khoản.....	10
2.1.1.2 Quản lý thông tin thủ thư.....	10
2.1.1.3 Quản lý sách	10
2.1.1.4 Quản lý tác giả.....	10
2.1.1.5 Quản lý giao dịch.....	10
2.1.1.6 Quản lý giao dịch và phí phạt trễ.....	11
2.2 Các đối tượng sử dụng.....	11
2.2.1 Quản trị hệ thống.....	11
2.2.2 Thủ thư (nhân viên).....	11
2.2.3 Độc giả (người mượn sách).....	11
2.3 Sơ đồ activity.....	11
2.4 Thiết kế CSDL MongoDB	12
2.4.1 Danh sách collections.....	12
2.4.1.1 Collection Account.....	12
2.4.1.2 Collection Staff.....	12
2.4.1.3 Collection Readers.....	12
2.4.1.4 Collection Books	12
2.4.1.5 Collection Categories	12
2.4.1.6 Collection Authors.....	12

2.4.1.7 Collection Transactions	12
2.4.1.8 Collection Loan	13
2.4.1.9 Collection Transaction.....	13
2.5.2 Mô hình hóa dữ liệu	13
2.5.3 Lược đồ dữ liệu (Document structure).....	13
2.5.3.1 Collection Account.....	13
2.5.3.2 Collection Staff.....	14
2.5.3.3 Collection Reader	14
2.5.3.4 Collection Books	14
2.5.3.5 Collection Categories	14
2.5.3.6 Collection Authors.....	14
2.5.3.7 Collection transactions	14
2.5.3.8 Collection Loan	15
CHƯƠNG 3: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	16
3.1 <i>Xây dựng cơ sở dữ liệu trên MongoDB</i>	<i>16</i>
3.1.1 Khởi tạo môi trường lưu trữ trên MongoDB Atlas	16
3.1.2 Cấu hình cluster và kết nối với công cụ truy vấn	19
3.2 <i>Kết nối MongoDB trong WinForms.....</i>	<i>20</i>
3.2.1 Tài thư viện ở Visual Studio	20
3.2.2 Kết nối .net với mongodb.....	21
3.3 <i>Các chức năng chính đã xây dựng.....</i>	<i>22</i>
3.4 <i>Giao diện và xử lý dữ liệu đẩy lên</i>	<i>23</i>
3.4.1 Đăng nhập	23
3.4.1.1 Giao diện	23
3.4.1.1 Code xử lý	23
3.4.2 Chức năng của quản trị viên.....	25
3.4.3 Chức năng của thủ thư.....	25
3.4.4 Quản lý thủ thư.....	26
3.4.4.1 Giao diện	26
3.4.1.2 Code xử lý	26
3.4.5 Quản lý người đọc.....	27
3.4.5.1 Giao diện	27
3.4.5.2 Code xử lý	28
3.4.6 Xem thông tin thủ thư	29
3.4.6.1 Giao diện	29
3.4.6.2 Code xử lý	29
3.4.7 Quản lý danh mục	31
3.4.7.1 Giao diện	31
3.4.7.1 Code xử lý	31
3.4.8 Quản lý tác giả.....	32
3.4.8.1 Giao diện	32
3.4.8.2 Code xử lý	33
3.4.9 Quản lý sách.....	34
3.4.9.1 Giao diện	34
3.4.9.2 Code xử lý	34
3.4.10 Quản lý mượn sách.....	36
3.4.10.1 Giao diện	36
3.4.10.1 Code xử lý	36

3.4.11 Quản lý giao dịch	37
3.4.11.1 Giao diện	37
3.4.11.2 Code xử lý	37
3.4.12 Thống kê số liệu của thư viện	39
3.4.12.1 Giao diện	39
3.4.12.2 Code xử lý	39
3.5 Một vài so sánh giữa MongoDB và SQL Server	40
3.5.1 Kết nối	40
3.5.2 Insert dữ liệu vào cơ sở dữ liệu	41
3.5.3 Update dữ liệu vào cơ sở dữ liệu	42
3.5.4 Delete dữ liệu vào cơ sở dữ liệu	42
PHẦN KẾT LUẬN	44
TÀI LIỆU THAM KHẢO	45

MỤC LỤC HÌNH ẢNH

Hình 2.1 Sơ đồ activity.....	11
Hình 2.2 Mối quan hệ giữa các collections.....	13
Hình 3.1 Tạo Organizations.....	16
Hình 3.2 Thêm thành viên vào Organization.....	16
Hình 3.3 Thêm thành viên vào Organization.....	17
Hình 3.4 Quản lý các truy cập trong Organization.....	17
Hình 3.5 Tạo Project.....	17
Hình 3.6 Thêm thành viên vào project.....	18
Hình 3.7 Tạo Cluster.....	18
Hình 3.8 Kết nối Cluster với công cụ truy vấn.....	19
Hình 3.9 Kết nối Cluster với công cụ truy vấn.....	19
Hình 3.10 Connection đã được kết nối với công cụ truy vấn.....	19
Hình 3.11 Mở NuGet Package.....	20
Hình 3.12 Cài đặt MongoDB Driver.....	20
Hình 3.13 Chỉnh sửa địa chỉ ip.....	21
Hình 3.14 Chọn mục tiêu kết nối.....	21
Hình 3.15 Sao chép chuỗi kết nối.....	21
Hình 3.16 Connection đã được kết nối.....	21
Hình 3.17 Các chức năng của quản trị viên.....	Error! Bookmark not defined.
Hình 3.18 Các chức năng của thủ thư.....	Error! Bookmark not defined.
Hình 3.19 giao diện đăng nhập.....	23
Hình 3.20 Các chức của quản trị viên.....	25
Hình 3.21 Các chức năng của thủ thư.....	25
Hình 3.21 Quản lý thủ thư.....	26
Hình 3.22 Quản lý người đọc.....	27
Hình 3.23 Xem thông tin của thủ thư.....	29
Hình 3.24 Xem thông tin của thủ thư.....	31
Hình 3.25 Quản lý tác giả.....	32
Hình 3.26 Quản lý book.....	34
Hình 3.27 Quản lý mượn sách.....	36
Hình 3.28 Quản lý mượn sách.....	37
Hình 3.29 Thông kê các số liệu.....	39

PHẦN MỞ ĐẦU

1. Tính cấp thiết của đề tài

Trong thời đại công nghệ số, việc chuyển đổi số các hoạt động quản lý đã trở thành xu hướng tất yếu nhằm nâng cao hiệu quả công việc và giảm thiểu sai sót do thao tác thủ công. Thư viện là nơi lưu trữ và cung cấp tri thức cũng cần được ứng dụng công nghệ để hiện đại hóa quy trình vận hành.

Hiện nay, nhiều hệ thống thư viện vẫn đang sử dụng cơ sở dữ liệu quan hệ (RDBMS) như SQL Server. Tuy nhiên, khi quy mô hệ thống mở rộng và dữ liệu ngày càng đa dạng, việc quản lý theo mô hình quan hệ bắt đầu xuất hiện một số bất cập, đặc biệt là trong các mối quan hệ phức tạp hoặc dữ liệu có tính linh hoạt cao. Trong bối cảnh đó, việc áp dụng cơ sở dữ liệu NoSQL đặc biệt là MongoDB, trở nên cần thiết để đáp ứng các yêu cầu mới về hiệu suất, mở rộng và khả năng tùy biến của hệ thống.

Do đó, việc nghiên cứu và chuyển đổi hệ thống thư viện từ SQL Server sang MongoDB là một đề tài có tính thực tiễn cao, giúp sinh viên nắm vững kiến thức về cơ sở dữ liệu, đồng thời tiếp cận được các công nghệ hiện đại đang được ứng dụng phổ biến trong thực tế.

2. Mục tiêu của đề tài

Đề tài được thực hiện với các mục tiêu chính sau:

Tìm hiểu và phân tích hệ thống thư viện đã có sẵn sử dụng SQL Server.

Thiết kế lại mô hình dữ liệu phù hợp với MongoDB trên cơ sở kế thừa hệ thống cũ.

Xây dựng phiên bản hệ thống sử dụng MongoDB làm cơ sở dữ liệu chính, tích hợp với giao diện WinForms (.NET).

So sánh ưu, nhược điểm giữa mô hình quan hệ và mô hình document trong bối cảnh ứng dụng thư viện.

3. Phạm vi nghiên cứu

Đề tài tập trung vào việc chuyển đổi cơ sở dữ liệu của hệ thống thư viện từ SQL Server sang MongoDB.

Phạm vi nghiên cứu bao gồm:

Phân tích dữ liệu của các bảng trong hệ thống thư viện cũ.

Thiết kế lại cấu trúc dữ liệu tương ứng trong MongoDB (theo hướng document-oriented).

Kết nối cơ sở dữ liệu MongoDB với ứng dụng WinForms để thực hiện các chức năng cơ bản như quản lý sách, người đọc, giao dịch mượn/trả và xử lý phí trễ hạn.

PHẦN NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG

1.1 Hệ thống thư viện

1.1.1 Tổng quan

Hệ thống thư viện là một giải pháp phần mềm được xây dựng nhằm quản lý toàn bộ hoạt động nghiệp vụ của một thư viện như: quản lý sách, người đọc, nhân viên, giao dịch mượn/trả và xử lý các khoản phí trễ hạn. Việc ứng dụng công nghệ thông tin vào thư viện giúp tiết kiệm thời gian, tăng độ chính xác và hỗ trợ người quản lý dễ dàng theo dõi hoạt động của thư viện.

Một hệ thống thư viện cơ bản thường có các chức năng sau:

Quản lý thông tin sách, danh mục, tác giả

Quản lý người dùng bao gồm nhân viên và người đọc

Theo dõi và quản lý việc mượn/trả sách

Ghi nhận các giao dịch tài chính như phí trễ, thanh toán

Phân quyền truy cập theo vai trò tài khoản (quản trị và thủ thư)

Trong thời đại chuyển đổi số, việc hiện đại hóa thư viện bằng phần mềm quản lý là cần thiết để đáp ứng nhu cầu sử dụng và tra cứu thông tin nhanh chóng, đặc biệt tại các thư viện trường học, cơ quan và thư viện công cộng.

1.1.2 Thực trạng

Hiện nay, hệ thống quản lý thư viện tại trường đang sử dụng cơ sở dữ liệu SQL Server 2016, kết hợp với một ứng dụng giao diện truyền thống, thiết kế đơn giản và đã lạc hậu so với yêu cầu sử dụng thực tế. Dữ liệu được lưu trữ theo mô hình quan hệ, thiếu sự linh hoạt khi xử lý các loại thông tin phức tạp như đa định dạng tài liệu, lịch sử mượn sách hay các mối quan hệ động giữa người dùng và tài nguyên thư viện.

Giao diện hệ thống hiện tại mang tính chất cơ bản, chưa thân thiện với người dùng, đặc biệt là sinh viên – những người cần tra cứu thông tin nhanh chóng và trực quan. Các chức năng tra cứu, thống kê, hoặc quản lý mượn – trả còn nhiều thao tác rườm rà, chưa có tính tự động hóa hoặc phân quyền rõ ràng cho các nhóm người dùng như thủ thư, sinh viên, giảng viên.

Ngoài ra, việc hệ thống phụ thuộc vào một kiến trúc cũ khiến khả năng mở rộng, tích hợp với các công nghệ mới (như API, nền tảng web hoặc mobile) trở nên khó khăn. Hiệu suất xử lý cũng không còn đáp ứng tốt khi số lượng đầu sách và người dùng tăng theo thời gian.

Từ những bất cập trên, có thể thấy hệ thống hiện tại không còn phù hợp với xu hướng công nghệ và nhu cầu thực tế trong môi trường giáo dục hiện đại. Điều này đặt ra yêu cầu cấp thiết cần xây dựng lại một hệ thống quản lý thư viện mới, hiện đại hơn, linh hoạt hơn, và dễ sử dụng hơn.

1.1.2.1 Ưu điểm

Mặc dù đã cũ, hệ thống hiện tại vẫn có một số ưu điểm đáng ghi nhận:

Mô hình quan hệ rõ ràng: SQL Server sử dụng mô hình quan hệ với các bảng được liên kết thông qua khóa chính – khóa ngoại, đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Dữ liệu chuẩn hóa: Các bảng được thiết kế theo chuẩn 3NF giúp loại bỏ dữ liệu dư thừa, dễ bảo trì và thuận tiện cho việc truy vấn bằng ngôn ngữ SQL.

Độ ổn định cao: SQL Server 2016 là phiên bản đã được chứng minh về độ tin cậy, đủ đáp ứng cho các hệ thống có quy mô vừa và nhỏ, như thư viện trường học.

Dễ tích hợp với WinForms: Hệ quản trị cơ sở dữ liệu này hoạt động mượt mà với các ứng dụng desktop viết bằng .NET, hỗ trợ kết nối dữ liệu đơn giản và nhanh chóng triển khai.

Tuy nhiên, những ưu điểm này không còn đủ để đáp ứng yêu cầu ngày càng cao của người dùng trong thời đại số. Việc đổi mới hệ thống trở nên cần thiết, không chỉ để cải thiện hiệu quả sử dụng mà còn để tạo nền tảng phát triển lâu dài, phù hợp với xu hướng công nghệ hiện đại.

1.1.2.1 Hạn chế

Bên cạnh những ưu điểm nhất định, hệ thống quản lý thư viện hiện tại cũng tồn tại nhiều hạn chế cả về mặt kỹ thuật lẫn trải nghiệm người dùng, cụ thể như sau:

Giao diện lỗi thời, thiếu thân thiện: Ứng dụng WinForms hiện tại sử dụng thiết kế đơn giản, thiếu tính trực quan và không phù hợp với thói quen sử dụng hiện đại.

Người dùng phải thao tác qua nhiều cửa sổ, menu lồng nhau, gây khó khăn khi tìm kiếm thông tin hoặc thực hiện các thao tác quản lý cơ bản như mượn – trả sách.

Khó mở rộng và tích hợp: Kiến trúc phần mềm dạng desktop truyền thống không phù hợp để tích hợp với các nền tảng hiện đại như ứng dụng web, mobile hay các dịch vụ API. Việc thêm tính năng mới hoặc kết nối với hệ thống khác (ví dụ: đăng nhập bằng tài khoản sinh viên, thông báo qua email, tích hợp thanh toán phạt) gặp nhiều trở ngại về kỹ thuật.

Phụ thuộc nặng vào mô hình quan hệ: Dữ liệu trong hệ thống hiện tại được lưu trữ dưới dạng bảng cố định, khiến việc xử lý các loại dữ liệu linh hoạt như lịch sử truy cập, hành vi người dùng, bình luận, đánh giá tài liệu... trở nên phức tạp. Mỗi thay đổi trong cấu trúc dữ liệu yêu cầu cập nhật nhiều bảng liên quan và điều chỉnh các truy vấn SQL thủ công.

Hiệu suất giảm khi dữ liệu tăng: SQL Server 2016 hoạt động ổn định ở mức dữ liệu vừa, nhưng khi quy mô người dùng và lượng tài nguyên tăng lên (ví dụ: hàng chục nghìn đầu sách, hàng nghìn lượt mượn mỗi tháng), hệ thống bắt đầu xuất hiện độ trễ, đặc biệt khi thực hiện các truy vấn thống kê hoặc lọc nâng cao.

Thiếu khả năng truy cập từ xa và đa nền tảng: Hệ thống hiện tại chỉ chạy trên máy cài đặt phần mềm, không thể truy cập qua trình duyệt hoặc thiết bị di động. Điều này gây bất tiện cho người dùng, đặc biệt trong bối cảnh học online hoặc làm việc từ xa trở nên phổ biến.

Khó bảo trì và cập nhật: Mỗi lần nâng cấp phần mềm hoặc sửa lỗi đều phải thao tác thủ công trên từng máy trạm. Việc không có nền tảng quản lý tập trung khiến việc bảo trì mất thời gian và dễ xảy ra lỗi không đồng nhất giữa các phiên bản.

1.2 Các công nghệ sử dụng

1.2.1 MongoDB

1.2.1.1 Giới thiệu về MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL dạng document-oriented sử dụng định dạng BSON (Binary JSON) để lưu trữ dữ liệu. Khác với cơ sở dữ liệu quan hệ (như SQL Server), MongoDB không lưu dữ liệu dưới dạng bảng mà sử dụng các document linh hoạt và có thể chứa dữ liệu lồng nhau (nested), phù hợp với các ứng dụng có cấu trúc dữ liệu động hoặc không đồng nhất.

MongoDB được phát triển bởi MongoDB Inc. và là một trong những cơ sở dữ liệu NoSQL phổ biến nhất hiện nay, nhờ khả năng mở rộng, hiệu năng cao và dễ tích hợp với các ngôn ngữ lập trình hiện đại như JavaScript, Python, C#, ...

1.2.1.2 MongoAtlas

MongoDB Atlas là một nền tảng cơ sở dữ liệu NoSQL được triển khai trên đám mây, cho phép người dùng dễ dàng tạo, quản lý và mở rộng các cluster MongoDB mà không cần tự cài đặt hay vận hành máy chủ. Với Atlas, dữ liệu được lưu trữ trực tuyến trên các hạ tầng như AWS, Azure hoặc Google Cloud, giúp đảm bảo tính sẵn sàng cao, bảo mật và khả năng mở rộng linh hoạt. Người dùng có thể kết nối đến MongoDB Atlas thông qua nhiều công cụ như MongoDB Compass hoặc plugin MongoDB for VSCode để truy vấn, chỉnh sửa và quản lý dữ liệu một cách thuận tiện.

1.2.1.3 Công cụ truy vấn dữ liệu

Để thao tác với dữ liệu trên MongoDB Atlas, người dùng có thể sử dụng các công cụ hỗ trợ như MongoDB Compass và MongoDB for VSCode. MongoDB Compass là giao diện đồ họa trực quan (GUI client), cho phép người dùng dễ dàng xem cấu trúc cơ sở dữ liệu, truy vấn tài liệu, phân tích schema và thực hiện các thao tác như thêm, sửa, xóa dữ liệu mà không cần viết mã lệnh phức tạp. Đây là lựa chọn phù hợp cho người mới bắt đầu hoặc các tác vụ kiểm thử nhanh. Trong khi đó, MongoDB for VSCode là một tiện ích mở rộng tích hợp trực tiếp trong môi trường phát triển Visual Studio Code, cho phép lập trình viên kết nối với MongoDB Atlas, viết truy vấn bằng cú pháp JavaScript hoặc shell, và tương tác với cơ sở dữ liệu ngay trong trình soạn mã. Nhờ sự linh hoạt của các công cụ này, việc phát triển và quản lý cơ sở dữ liệu MongoDB trở nên nhanh chóng, hiệu quả và dễ tiếp cận hơn.

1.2.1.4 Dữ liệu phi cấu trúc

Dữ liệu phi cấu trúc là loại dữ liệu không tuân theo mô hình bảng gồm hàng và cột như trong cơ sở dữ liệu quan hệ. Nó không có một khuôn mẫu (schema) cố định và thường xuất hiện dưới các định dạng linh hoạt như văn bản, hình ảnh, tệp JSON, XML, hoặc các loại tài liệu tự do.

Sự xuất hiện của dữ liệu phi cấu trúc gắn liền với sự phát triển mạnh mẽ của các ứng dụng web, mạng xã hội và hệ thống có tính chất dữ liệu thay đổi linh hoạt theo thời gian. Trong bối cảnh đó, các hệ quản trị cơ sở dữ liệu truyền thống dần bộc lộ hạn chế về khả năng mở rộng và thích ứng với dữ liệu động.

Các hệ cơ sở dữ liệu NoSQL tiêu biểu là MongoDB, ra đời nhằm đáp ứng nhu cầu lưu trữ và xử lý dữ liệu phi cấu trúc. MongoDB cho phép lưu trữ dữ liệu dưới dạng tài liệu (document), có khả năng mở rộng linh hoạt, không yêu cầu cấu trúc cố định, và hỗ trợ tổ chức dữ liệu lồng nhau một cách tự nhiên.

Dữ liệu phi cấu trúc đóng vai trò quan trọng trong các hệ thống hiện đại, nơi mà sự linh hoạt, khả năng mở rộng và tốc độ truy xuất là những yêu cầu ưu tiên hàng đầu.

1.2.1.5 So sánh dữ liệu có cấu trúc và phi cấu trúc

Tiêu chí	Dữ liệu có cấu trúc	Dữ liệu phi cấu trúc
Mô hình tổ chức	Hàng – cột trong bảng	Dạng linh hoạt: văn bản, JSON, XML, ...
Schema (cấu trúc dữ liệu)	Cố định, phải định nghĩa trước	Không cố định, có thể thay đổi theo nhu cầu
Lưu trữ	Trong cơ sở dữ liệu quan hệ (RDBMS)	Trong hệ thống NoSQL, file, hệ thống phân tán
Tính linh hoạt	Hạn chế khi thay đổi cấu trúc	Cao, dễ thay đổi, mở rộng
Tốc độ truy vấn dữ liệu	Nhanh và ổn định với dữ liệu chuẩn hóa	Phù hợp với truy vấn đơn giản, nhanh gọn
Khả năng mở rộng	Theo chiều dọc (nâng cấp phần cứng)	Theo chiều ngang (thêm máy chủ mới)
Loại dữ liệu phù hợp	Dữ liệu đồng nhất, nghiệp vụ rõ ràng	Dữ liệu đa dạng, linh hoạt, thay đổi thường xuyên

1.2.2 WinForms (.NET)

1.2.2.1 Giới thiệu về Winform (.NET)

WinForms (Windows Forms) là một thư viện giao diện người dùng (UI – User Interface) trong nền tảng .NET Framework của Microsoft. Đây là một công cụ phổ biến được sử dụng để xây dựng các ứng dụng máy tính chạy trên hệ điều hành Windows với giao diện đồ họa (GUI – Graphical User Interface).

WinForms cung cấp sẵn các thành phần trực quan như nút bấm (Button), hộp văn bản (TextBox), bảng dữ liệu (DataGridView), menu, v.v., giúp lập trình viên dễ dàng tạo ra các cửa sổ làm việc thân thiện với người dùng. So với các công nghệ giao diện khác như WPF hay UWP, WinForms đơn giản, dễ học và phù hợp với các ứng dụng quản lý nội bộ quy mô vừa và nhỏ.

1.2.2.2 C# Programming

C# là ngôn ngữ lập trình chính được sử dụng trong WinForms, do Microsoft phát triển. C# hỗ trợ lập trình hướng đối tượng, có cú pháp dễ hiểu, tương tự như Java và C++, đồng thời tích hợp chặt chẽ với thư viện .NET Framework.

Trong dự án này C# được sử dụng để:

Xây dựng giao diện người dùng (UI) với WinForms

Xử lý các sự kiện tương tác (như nút nhấn, nhập liệu)

Kết nối với cơ sở dữ liệu (SQL Server hoặc MongoDB)

Thực hiện các chức năng nghiệp vụ như thêm, sửa, xóa, tìm kiếm dữ liệu.

1.2.3 Kiến trúc ứng dụng

1.2.3.1 Mô hình 3 lớp (Three-Tier Architecture)

Mô hình ba lớp là một trong những kiến trúc phần mềm phổ biến và được sử dụng rộng rãi trong các ứng dụng quản lý. Cấu trúc ba lớp bao gồm:

Lớp giao diện (Presentation Layer): Giao diện tương tác với người dùng. Trong project lớp này được xây dựng bằng WinForms (.NET), bao gồm các biểu mẫu như quản lý sách, người đọc, giao dịch,...

Lớp xử lý nghiệp vụ (Business Logic Layer): Chứa các quy tắc xử lý, tính toán, kiểm tra hợp lệ dữ liệu (ví dụ: kiểm tra thông tin khi mượn sách, tính phí trễ,...).

Lớp truy xuất dữ liệu (Data Access Layer): Đảm nhận việc kết nối, truy vấn và cập nhật dữ liệu từ cơ sở dữ liệu (có thể là SQL Server hoặc MongoDB).

Việc chia ứng dụng thành ba lớp giúp tăng tính phân tách trách nhiệm, dễ dàng nâng cấp từng phần mà không ảnh hưởng đến toàn bộ hệ thống.

1.2.3.2 Mô hình Client–Server

Bên cạnh mô hình ba lớp, ứng dụng còn tuân theo nguyên lý của mô hình client–server, trong đó:

Client: Là ứng dụng WinForms chạy trên máy người dùng, thực hiện các thao tác nhập liệu, tìm kiếm, hiển thị dữ liệu.

Server: Là nơi lưu trữ cơ sở dữ liệu và xử lý các truy vấn. Trong hệ thống hiện tại, server có thể là MongoDB (NoSQL).

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu hệ thống

2.1.1 Yêu cầu chức năng

Hệ thống quản lý thư viện cần đáp ứng các chức năng chính sau:

2.1.1.1 Quản lý tài khoản

Tài khoản quản trị hệ thống có thể đăng nhập vào để quản lý thông tin và tài khoản các thủ thư

Tài khoản thủ thư (nhân viên) có thể đăng nhập vào hệ thống với quyền quản lý sách, tác giả, thẻ loại, độc giả và giao dịch mượn trả. Ngoài ra thủ thư có thể thay đổi mật khẩu tài khoản

2.1.1.2 Quản lý thông tin thủ thư

Quản trị hệ thống có thể thêm mới thủ thư, xóa thủ thư và chỉnh sửa thông tin thủ thư.

2.1.1.3 Quản lý sách

Cho phép thêm mới, chỉnh sửa, xóa thông tin sách.

Cho phép tìm kiếm sách theo nhiều tiêu chí: tên sách, tác giả, thẻ loại, năm xuất bản

Quản lý loại sách

Quản lý danh sách các thẻ loại sách.

Cho phép thêm mới, chỉnh sửa hoặc xóa các thẻ loại khi cần thiết.

Gắn thẻ loại với các đầu sách để phục vụ tìm kiếm và phân loại.

2.1.1.4 Quản lý tác giả

Lưu trữ thông tin về các tác giả: họ tên, quốc tịch, tiểu sử, v.v.

Cho phép gán nhiều tác giả cho một cuốn sách nếu cần.

Hỗ trợ việc tìm kiếm và thống kê sách theo tác giả.

2.1.1.5 Quản lý giao dịch

Ghi nhận thông tin mượn sách: độc giả, sách, ngày mượn, ngày dự kiến trả.

Cập nhật trạng thái khi độc giả trả sách.

Cho phép xem lịch sử giao dịch theo từng người dùng.

Tự động kiểm tra và cập nhật trạng thái quá hạn.

2.1.1.6 Quản lý giao dịch và phí phạt trễ

Hệ thống tự động tính phí phạt khi độc giả trả sách sau hạn định.

Quản lý danh sách các khoản phí phạt, số ngày trễ và số tiền tương ứng.

2.2 Các đối tượng sử dụng

2.2.1 Quản trị hệ thống

Quản lý thủ thư, xem, xóa và cập nhật thông tin của thủ thư.

2.2.2 Thủ thư (nhân viên)

Quản lý sách, tác giả, thể loại.

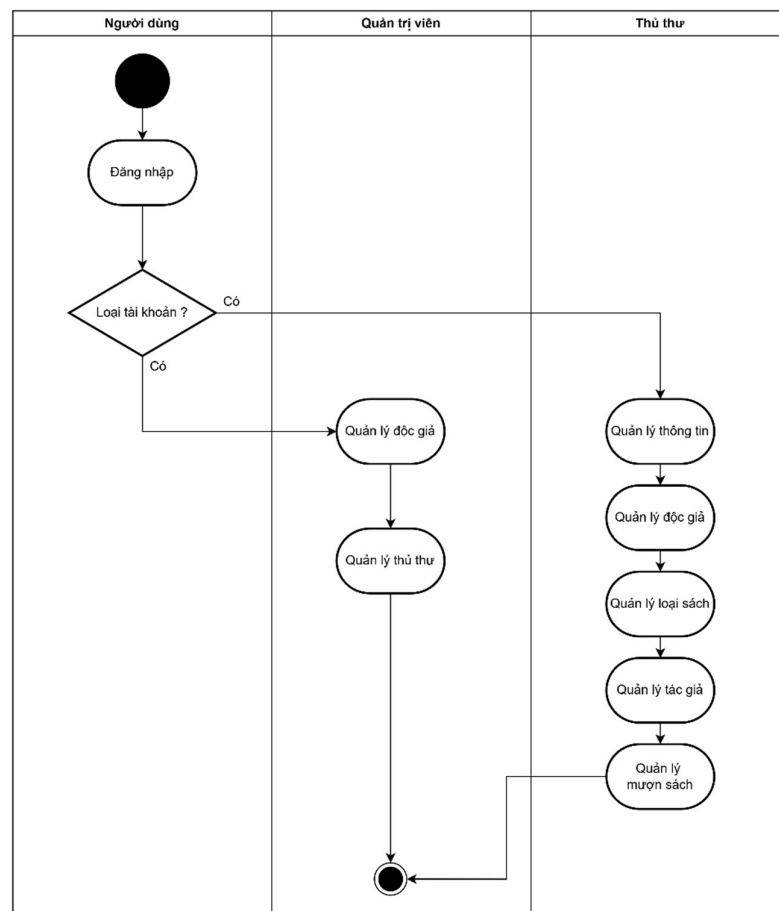
Quản lý độc giả, xem và cập nhật thông tin mượn/trả sách.

Tính phí phạt cho các trường hợp trễ hạn.

Tạo và quản lý tài khoản nhân viên khác.

2.2.3 Độc giả (người mượn sách)

2.3 Sơ đồ activity



Hình 2.1 Sơ đồ activity

2.4 Thiết kế CSDL MongoDB

2.4.1 Danh sách collections

Hệ thống sử dụng cơ sở dữ liệu dạng NoSQL (ví dụ MongoDB), các đối tượng được lưu trữ dưới dạng collections (bộ sưu tập). Mỗi collection tương ứng với một đối tượng thực thể trong hệ thống, cụ thể như sau:

2.4.1.1 Collection Account

Lưu trữ thông tin tài khoản người dùng gồm cả độc giả và nhân viên thư thư.

Các trường dữ liệu phổ biến: username, password (được mã hóa), role (độc giả/thư thư), email, thông tin cá nhân, trạng thái hoạt động.

2.4.1.2 Collection Staff

Lưu thông tin chi tiết của nhân viên thư thư, bao gồm: họ tên, ngày sinh, chức vụ, số điện thoại, thông tin đăng nhập liên kết đến Collection Account.

2.4.1.3 Collection Readers

Lưu thông tin chi tiết của người đọc, bao gồm: họ tên, ngày sinh, số điện thoại, nghề nghiệp, ...

2.4.1.4 Collection Books

Lưu trữ thông tin các đầu sách trong thư viện.

Dữ liệu bao gồm: tên sách, tác giả, thể loại, năm xuất bản

2.4.1.5 Collection Categories

Quản lý danh sách các thể loại sách như: Khoa học, Văn học, Lịch sử, Công nghệ,...

Mỗi bản ghi bao gồm mã thể loại, tên thể loại, mô tả thể loại.

2.4.1.6 Collection Authors

Lưu trữ thông tin chi tiết về tác giả sách.

Bao gồm: họ tên, quốc tịch, năm sinh, tiểu sử, danh sách sách liên kết.

2.4.1.7 Collection Transactions

Ghi nhận các giao dịch mượn – trả sách giữa độc giả và thư viện.

Dữ liệu bao gồm: mã giao dịch, mã độc giả, mã sách, ngày mượn, ngày trả dự kiến, ngày trả thực tế, trạng thái.

2.4.1.8 Collection Loan

Lưu trữ thông tin các lượt mượn và trả sách của người dùng.

Dữ liệu bao gồm: mã lượt mượn, tên đăng nhập của người đọc, mã sách, ngày mượn, ngày hết hạn mượn, ngày trả thực tế và trạng thái mượn.

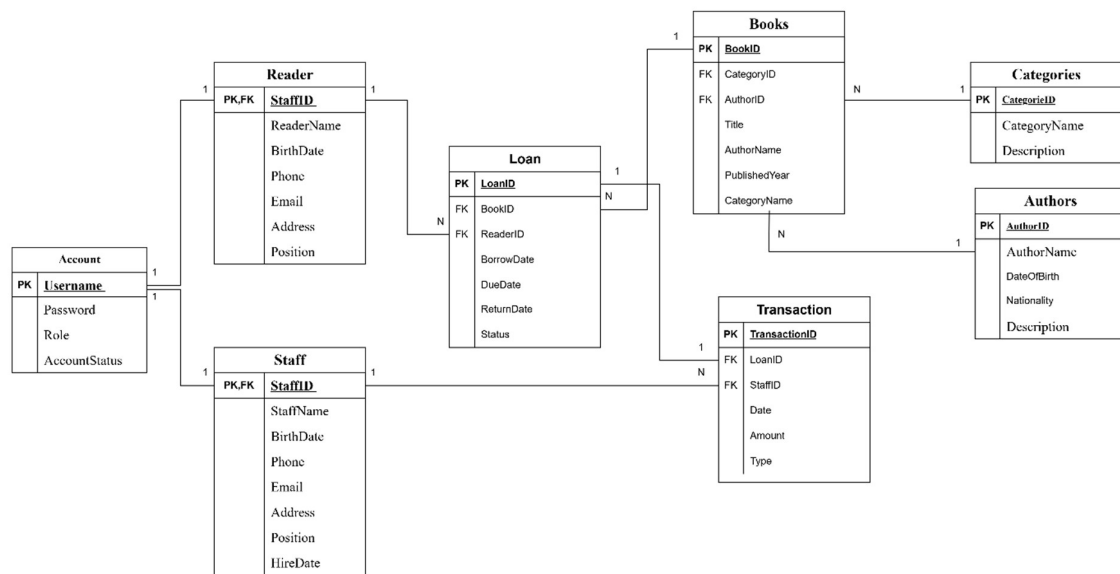
Trạng thái có thể là: BORROWED (đang mượn), RETURNED (đã trả), hoặc OVERDUE (trả trễ hạn).

2.4.1.9 Collection Transaction

Ghi nhận các giao dịch tài chính giữa thư viện và người dùng hoặc nhân viên.

Dữ liệu bao gồm: mã lượt mượn (loanId), tên đăng nhập của nhân viên thực hiện (staffUsername), ngày giao dịch, loại giao dịch (Payment, Fine, v.v.), và số tiền liên quan.

2.5.2 Mô hình hóa dữ liệu



Hình 2.2 Mối quan hệ giữa các collections

2.5.3 Lược đồ dữ liệu (Document structure)

2.5.3.1 Collection Account

```
{
  _id: ObjectId,    // MongoDB tự sinh ID
  username: String, // Tên người dùng (duy nhất)
  password: String, // Mật khẩu đã mã hóa
  role: String,     // "Reader" hoặc "Staff"
  accountStatus: String // "Active", "Inactive"
}
```

2.5.3.2 Collection Staff

```
{
  _id: ObjectId,
  username: String,
  staffName: String,
  phone: String,
  email: String,
  address: String,
  position: String,
  hireDate: ISODate
}
```

2.5.3.3 Collection Reader

```
{
  _id: ObjectId,
  username: String,
  readerName: String,
  phone: String,
  email: String,
  address: String,
  position: String,
  hireDate: ISODate
}
```

2.5.3.4 Collection Books

```
{
  _id: ObjectId,
  title: String,
  publishedYear: Number,
  publisher: String
}
```

2.5.3.5 Collection Categories

```
{
  _id: ObjectId,
  name: String,
  description: String
}
```

2.5.3.6 Collection Authors

```
{
  _id: ObjectId,
  name: String,
  bio: String
}
```

2.5.3.7 Collection transactions

```
// transactions
{
  _id: ObjectId,
  loanId: ObjectId,
```

```
employeeId: ObjectId,  
date: ISODate,  
amount: Number,  
type: String  
}
```

2.5.3.8 Collection Loan

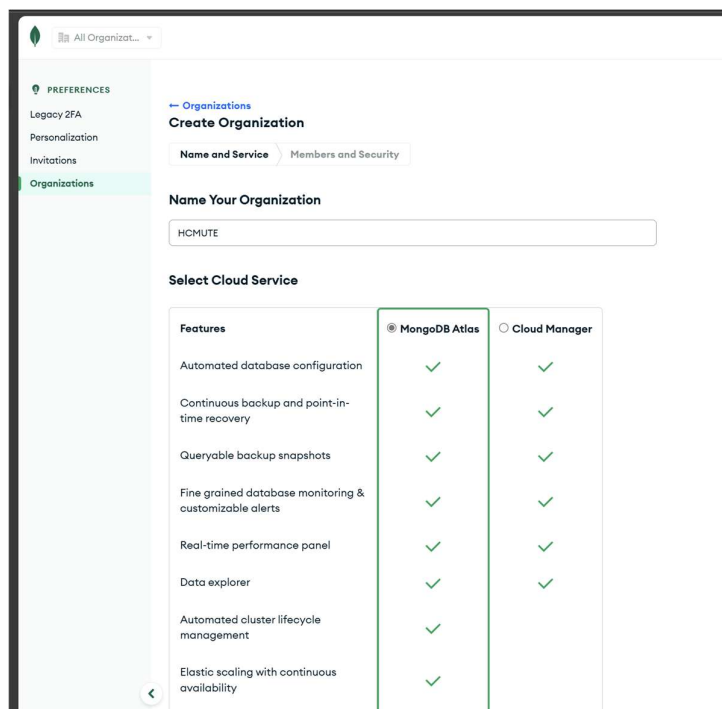
```
// Loans  
{  
  _id: ObjectId(),  
  readerId: ObjectId(),  
  bookId: ObjectId(),  
  borrowdate: ISODate(),  
  dueDate: ISODate(),  
  returndate: ISODate(),  
  status: String  
}
```

CHƯƠNG 3: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

3.1 Xây dựng cơ sở dữ liệu trên MongoDB

3.1.1 Khởi tạo môi trường lưu trữ trên MongoDB Atlas

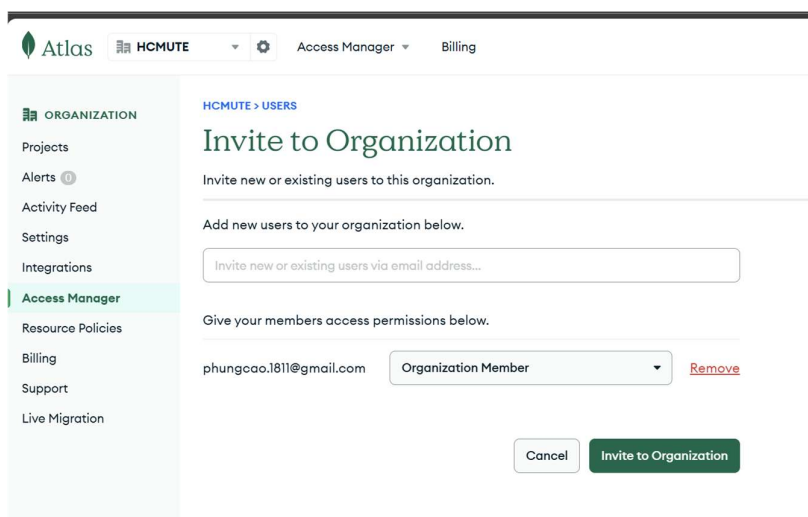
Tạo Organizations Đặt tên cho Organization là “HCMUTE-ADDB”



Features	Ⓜ MongoDB Atlas	○ Cloud Manager
Automated database configuration	✓	✓
Continuous backup and point-in-time recovery	✓	✓
Queryable backup snapshots	✓	✓
Fine grained database monitoring & customizable alerts	✓	✓
Real-time performance panel	✓	✓
Data explorer	✓	✓
Automated cluster lifecycle management	✓	
Elastic scaling with continuous availability	✓	

Hình 3.1 Tạo Organizations

Mời các thành viên trong nhóm vào Organization



Atlas HCMUTE Access Manager Billing

ORGANIZATION

Projects

Alerts

Activity Feed

Settings

Integrations

Access Manager

Resource Policies

Billing

Support

Live Migration

HCMUTE > USERS

Invite to Organization

Invite new or existing users to this organization.

Add new users to your organization below.

Invite new or existing users via email address...

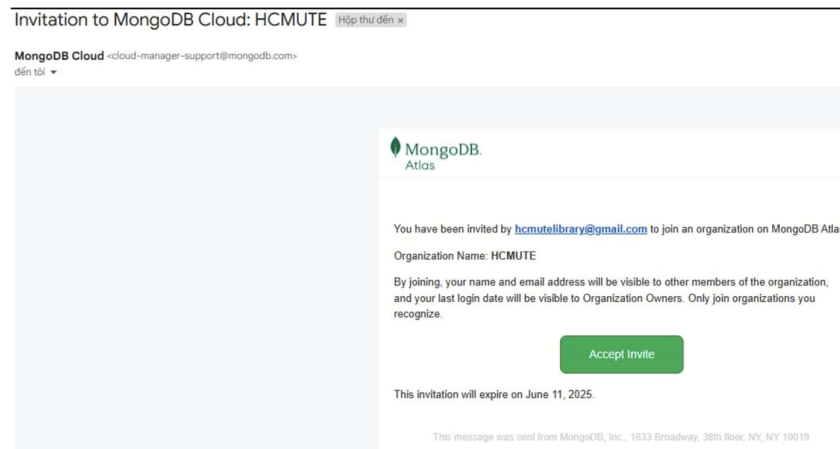
Give your members access permissions below.

phungcao.1811@gmail.com Organization Member Remove

Cancel Invite to Organization

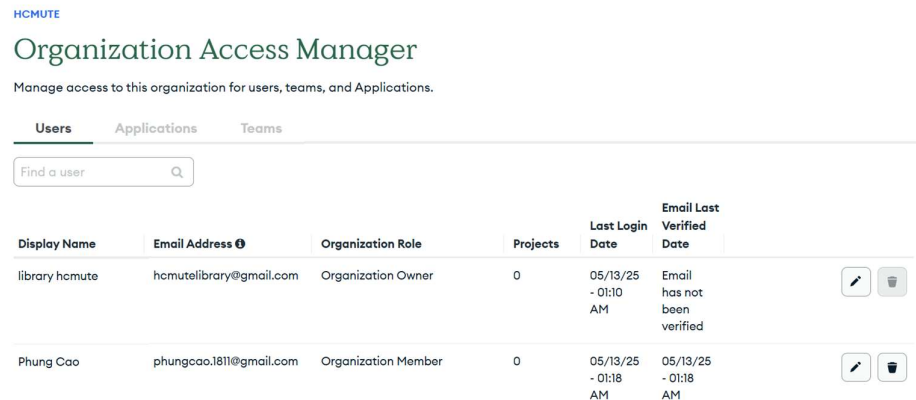
Hình 3.2 Thêm thành viên vào Organization

Thành viên được mời sẽ nhận mail và chấp nhận



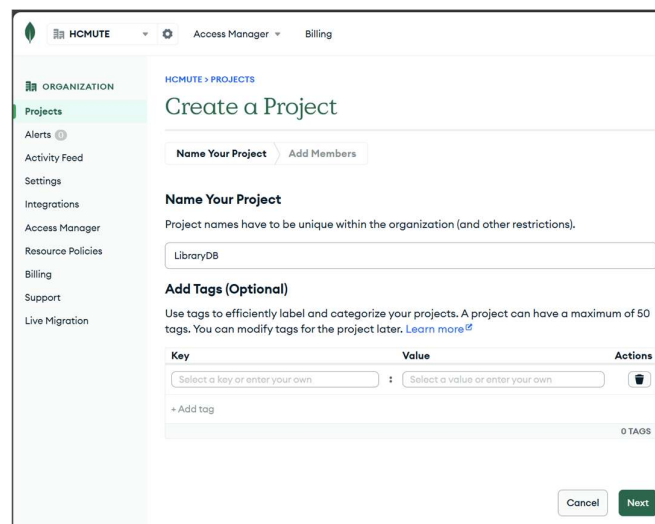
Hình 3.3 Thêm thành viên vào Organization

Quản lý các truy cập trong Organization



Hình 3.4 Quản lý các truy cập trong Organization

Tạo Project và đặt tên Project là “LibraryDB”



Hình 3.5 Tạo Project

Thêm thành viên vào project

HCMUTE > PROJECTS

Create a Project

✓ Name Your Project Add Members

Add Members and Set Permissions

Invite new or existing users via email address...

Give your members access permissions below.

hcmutelibrary@gmail.com
(you)

Project Owner

phungcao.1811@gmail.com

Project Read Only

[Remove](#)

Hình 3.6 Thêm thành viên vào project

Tạo và đặt tên cho clusters

Configurations

Name

You cannot change the name once the cluster is created.

ClusterNgocPhung

Provider



Region

★ Hong Kong (ap-east-1) ★

★ Recommended ⓘ

🌿 Low carbon emissions ⓘ

Tag (optional)

Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

Select or enter key

:

Select or enter value

Quick setup

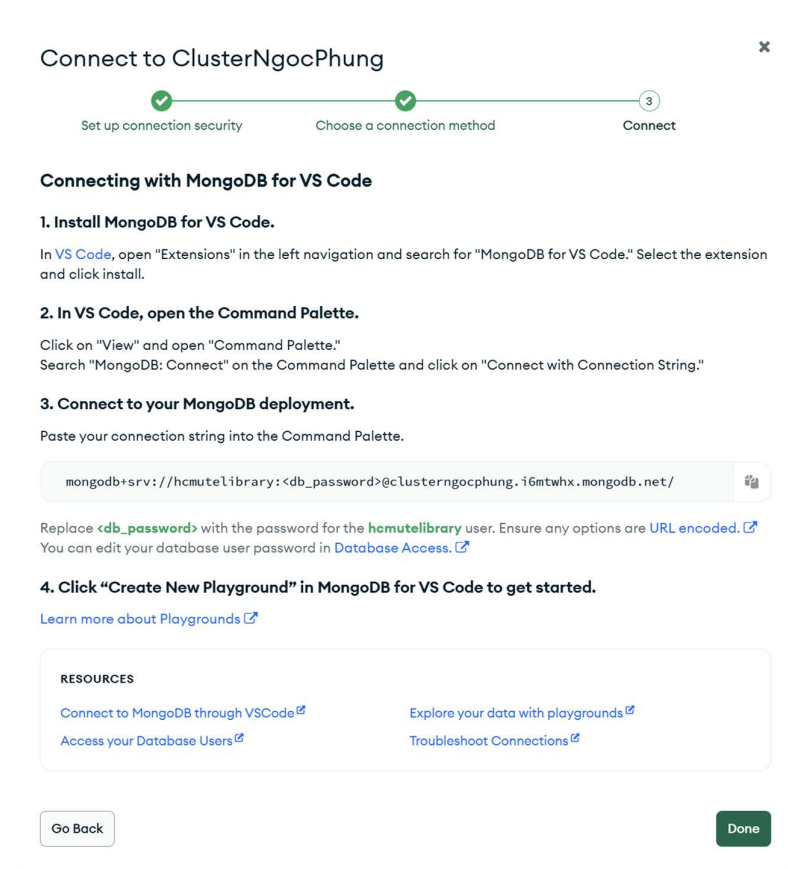
☒ Automate security setup ⓘ

☒ Preload sample dataset ⓘ

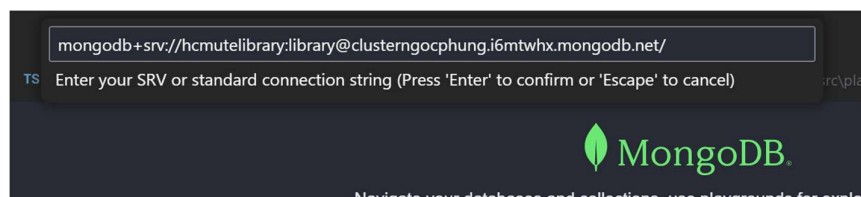
Hình 3.7 Tạo Cluster

3.1.2 Cấu hình cluster và kết nối với công cụ truy vấn

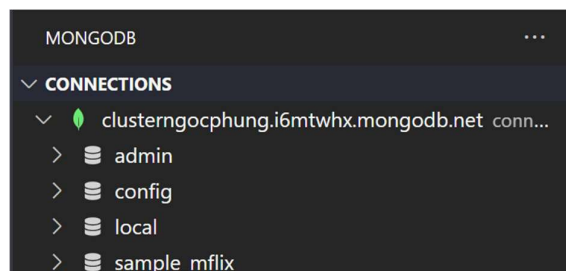
Kết nối Cluster với công cụ truy vấn



Hình 3.8 Kết nối Cluster với công cụ truy vấn



Hình 3.9 Kết nối Cluster với công cụ truy vấn



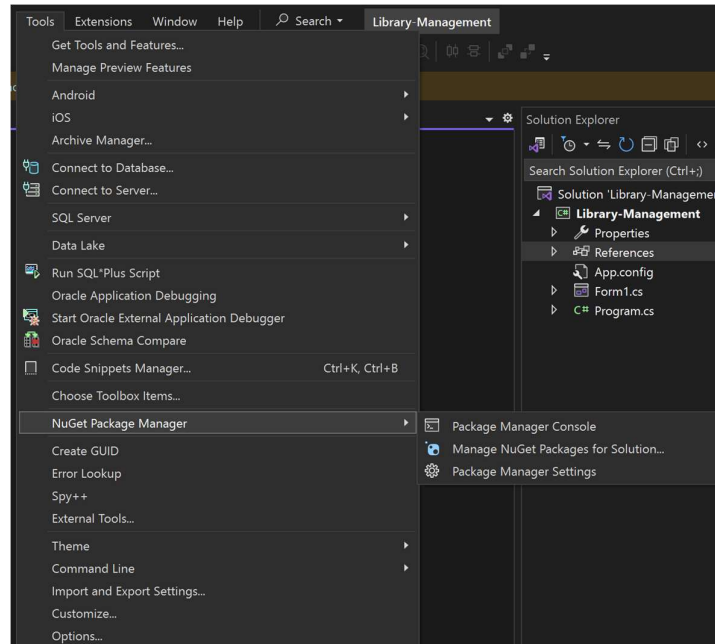
Hình 3.10 Connection đã được kết nối với công cụ truy vấn

Các thành viên khác muốn kết nối với cluster có thể thực hiện lại các bước trên để có thể kết nối giữa cluster và công cụ truy vấn trên máy cá nhân.

3.2 Kết nối MongoDB trong WinForms

3.2.1 Tải thư viện ở Visual Studio

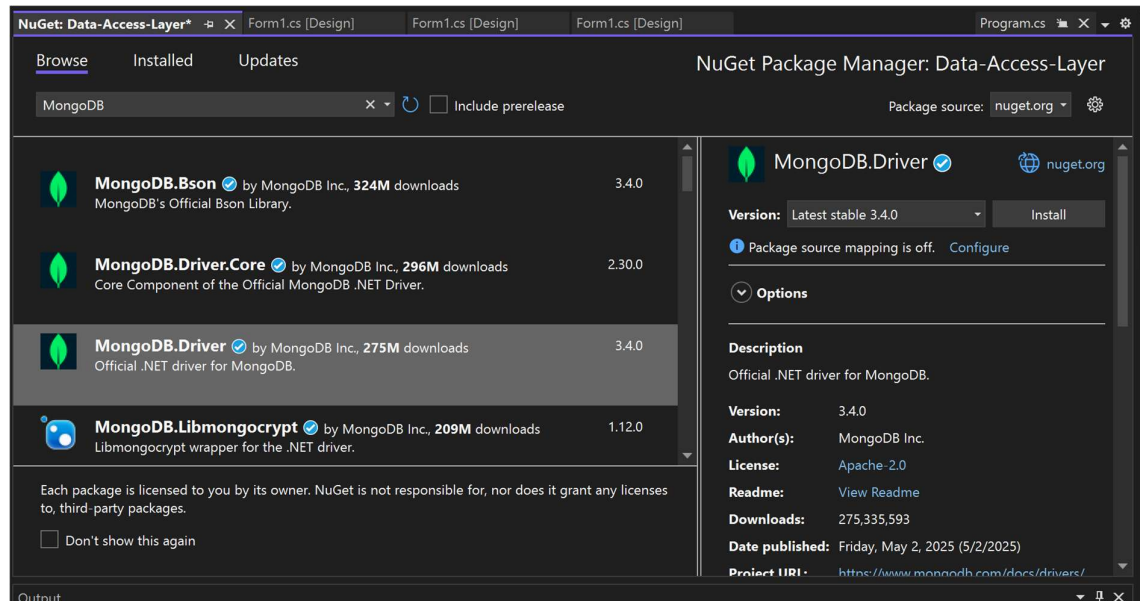
Khi tạo project vào tool → Manage NuGet Package



Hình 3.11 Mở NuGet Package

Chọn tab Browse và tìm MongoDB

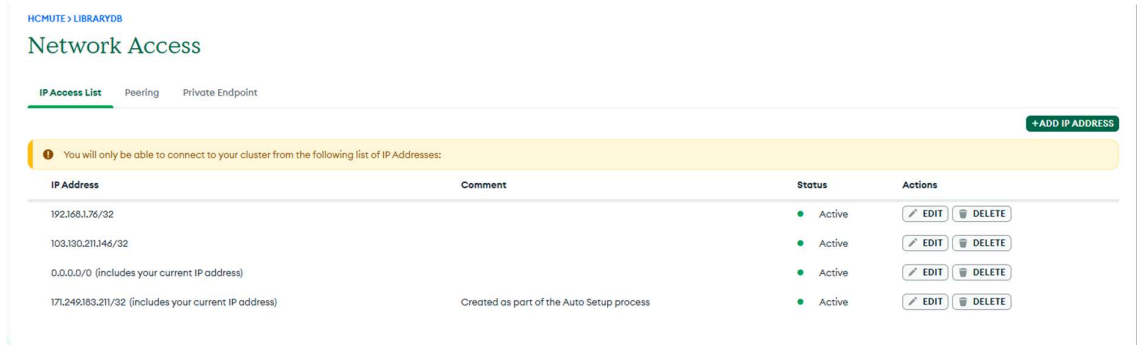
Manage NuGet Package



Hình 3.12 Cài đặt MongoDB Driver

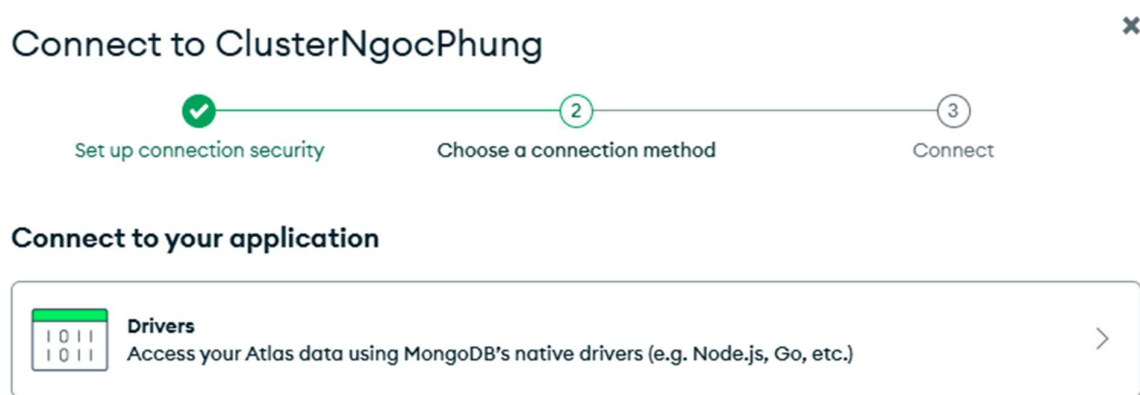
3.2.2 Kết nối .net với mongodb

Vào phần “Network Access” của mongodb atlas thêm ip “0.0.0.0/0” để cho phép tất cả địa chỉ ip kết nối với mongo atlas



Hình 3.13 Chính sửa địa chỉ ip

Chọn “Drivers” và copy link kết nối với servers



Hình 3.14 Chọn mục tiêu kết nối

Use this connection string in your application

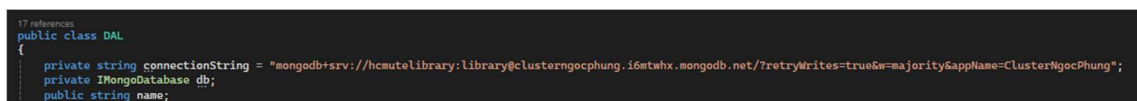
☐ View full code sample

```
mongodb+srv://hcmutelibrary:<db_password>@clusterngocphung.i6mtwhx.mongodb.net/?retryWrites=true&w=majority&appName=ClusterNgocPhung
```

Replace **<db_password>** with the password for the **hcmutelibrary** database user. Ensure any option params are

Hình 3.15 Sao chép chuỗi kết nối

Vì mongodb là cơ sở dữ liệu phi cấu trúc, nên không kết nối qua server explore của visual studio code, connect bằng 1 chuỗi string thông qua



Hình 3.16 Connection đã được kết nối

3.3 Các chức năng chính đã xây dựng

```
// Phụng
use("LibraryDB");
db.createCollection("account");
db.account.createIndex({ username: 1 }, { unique: true });
db.createCollection("staff");
db.staff.createIndex({ username: 1 }, { unique: true });
db.createCollection("reader");
db.reader.createIndex({ username: 1 }, { unique: true });
//Quỳnh
db.createCollection("author");
db.author.createIndex({ authorId: 1 }, { unique: true });
db.author.createIndex({ authorName: 1 }, { unique: true });
db.createCollection("book");
db.book.createIndex({ bookId: 1 }, { unique: true });
db.createCollection("category");
db.category.createIndex({ categoryId: 1 }, { unique: true });
// Tiến
db.createCollection("loan");
db.loan.createIndex({ readerId: 1 })
db.loan.createIndex({ status: 1 })
db.loan.createIndex({ bookId: 1 })
db.createCollection("transaction");
db.transaction.createIndex({ loanId: 1 })
db.transaction.createIndex({ employeeId: 1 })
```

Đầu tiên, cơ sở dữ liệu được lựa chọn để làm việc là LibraryDB thông qua lệnh `use("LibraryDB")`. Sau đó, các collection cần thiết trong hệ thống quản lý thư viện được tạo ra bằng lệnh `db.createCollection(...)`, bao gồm: `account`, `staff`, `reader`, `author`, `book`, `category`, `loan` và `transaction`.

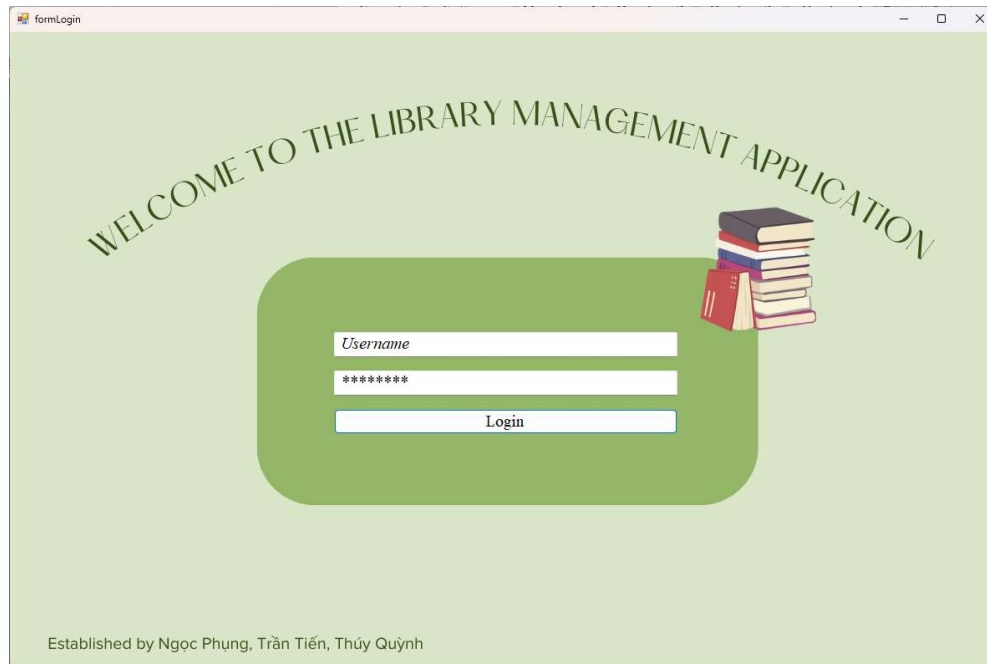
Đối với các collection chứa thông tin định danh như `account`, `staff` và `reader`, hệ thống tạo chỉ mục duy nhất (unique index) trên trường `username` nhằm đảm bảo rằng mỗi người dùng chỉ có một tài khoản duy nhất, đồng thời tăng tốc độ truy vấn theo `username`. Tương tự, collection `author` được tạo với hai chỉ mục duy nhất trên trường `authorId` và `authorName`, mục đích là tránh trùng lặp ID và tên tác giả. Collection `book` và `category` cũng được đánh chỉ mục duy nhất lần lượt trên `bookId` và `categoryId` để đảm bảo mỗi sách và thể loại đều có mã định danh riêng biệt, không bị trùng.

Tiếp theo, collection `loan` được đánh chỉ mục trên ba trường: `readerId`, `status`, và `bookId`. Việc tạo các chỉ mục này giúp tối ưu truy vấn khi cần tìm các đơn mượn theo người dùng, trạng thái mượn (ví dụ: “BORROWED” hay “RETURNED”) và mã sách. Cuối cùng, collection `transaction` được tạo cùng hai chỉ mục trên `loanId` và `employeeId` để phục vụ truy vấn nhanh các giao dịch liên quan đến đơn mượn cụ thể và nhân viên xử lý giao dịch.

3.4 Giao diện và xử lý dữ liệu đẩy lên

3.4.1 Đăng nhập

3.4.1.1 Giao diện



Hình 3.19 giao diện đăng nhập

3.4.1.1 Code xử lý

Để kiểm tra thông tin đăng nhập của người dùng, sự kiện `buttonLogin_Click` sẽ gọi đến hàm `CheckLogin` trong lớp `account` thuộc tầng Business Logic Layer (BLL). Mục đích là so sánh username và password mà người dùng nhập với dữ liệu lưu trữ trong MongoDB. Nội dung hàm `buttonLogin_Click` được trình bày như sau:

```
private void buttonLogin_Click(object sender, EventArgs e)
{
    string username = textBoxUserName.Text;
    string password = textBoxPassWord.Text;
    try
    {
        string[] result = account.CheckLogin(username, password);
        if (result != null)
        {
            string role = result[0];
            string status = result[1];
            if (status != "Active")
            {
                MessageBox.Show("⚠ Tài khoản của bạn đang bị khóa hoặc tạm ngưng!");
                return;
            }
        }
        else
        {
            if (role == "Admin")
            {
                formMenuAdmin formmenuadmin = new formMenuAdmin();
                formmenuadmin.Show();
                this.Hide();
            }
        }
    }
}
```

```

        else if (role == "Staff")
        {
            formMenuStaff formmenustaff = new formMenuStaff();
            formmenustaff.Show();
            this.Hide();
        }
        else
        {
            MessageBox.Show("Không có quyền truy cập");
            Session.Username = username;
            Session.Role = role;
        }
    }
    else
    {
        MessageBox.Show("❌ Sai tài khoản hoặc mật khẩu!");
    }
}
catch (Exception ex)
{
    MessageBox.Show("⚠️ Lỗi kết nối MongoDB: " + ex.Message, "Lỗi hệ thống");
}
}

```

Hàm CheckLogin trong lớp account sẽ thực hiện việc truy xuất dữ liệu từ MongoDB thông qua hàm Find trong lớp Data Access Layer (DAL). Dưới đây là nội dung hàm CheckLogin:

```

public string[] CheckLogin(string username, string password)
{
    string filter = $"{{ \"username\": \"{username}\", \" +
        $\"password\": \"{password}\" }}";

    var result = dal.Find("account", filter);

    if (result.Count > 0)
    {
        string role = result[0]["role"].AsString;
        string status = result[0]["status"].AsString;
        return new string[] { role, status };
    }
    return null;
}

```

Hàm Find ở lớp DAL có nhiệm vụ kết nối đến MongoDB và truy xuất dữ liệu theo điều kiện lọc được truyền vào. Nội dung cụ thể của hàm Find như sau:

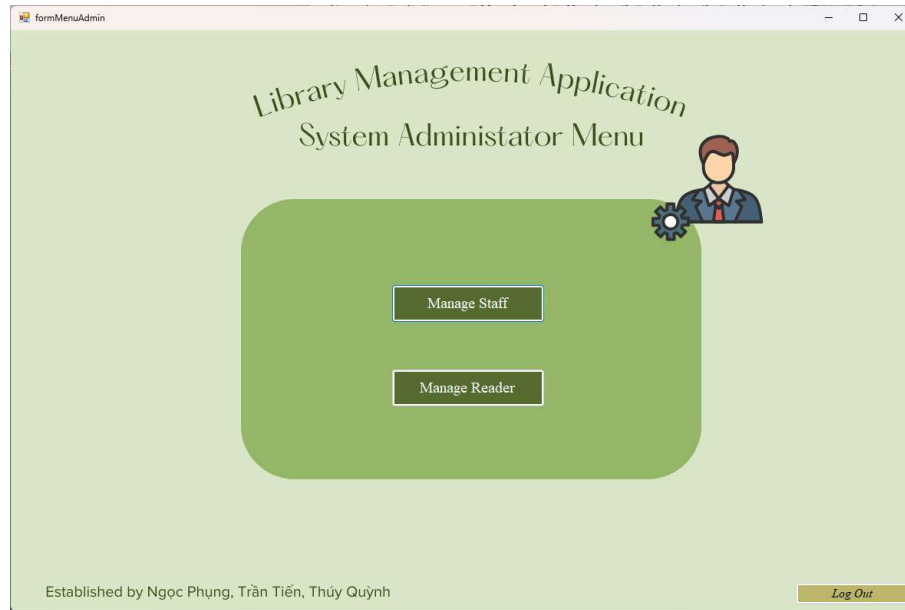
```

public List<BsonDocument> Find(string collectionName, string filterString)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    var filter = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(filterString);
    return collection.Find(filter).ToList();
}

```

3.4.2 Chức năng của quản trị viên

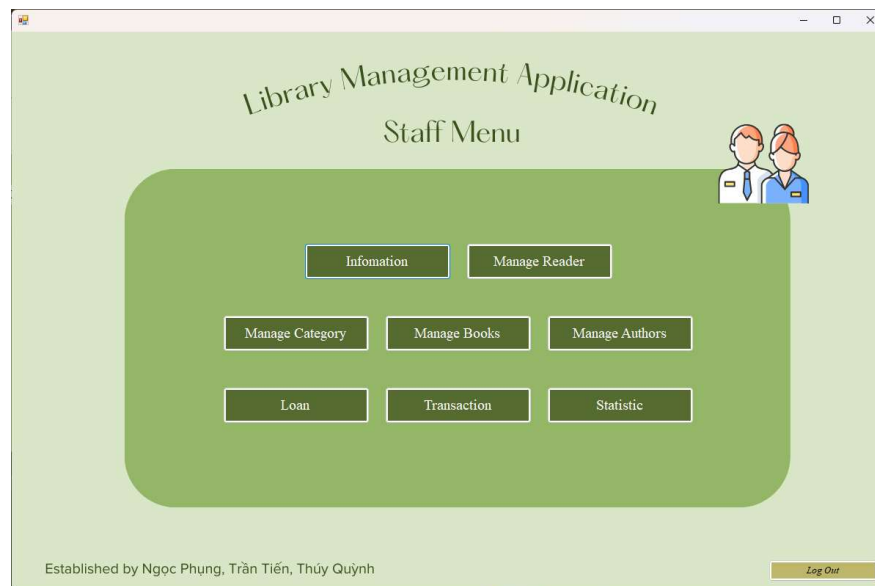
Giao diện chức năng của quản trị viên



Hình 3.20 Các chức của quản trị viên

3.4.3 Chức năng của thủ thư

Giao diện chức năng của thủ thư



Hình 3.21 Các chức năng của thủ thư

3.4.4 Quản lý thủ thư

Quản lý nhân viên (Manage Staff): Cho phép thêm, sửa, xóa thông tin tài khoản của các nhân viên thư viện. Ngoài ra, quản trị viên có thể gán quyền truy cập, theo dõi lịch sử hoạt động và đảm bảo phân quyền phù hợp theo vai trò.

3.4.4.1 Giao diện

username	staffName	birthdate	phone	email
staff_ngocphung	Cao Thị Ngọc Ph...	2019-07-13T17:0...	0363181036	phungcao.1811...
staff_tien	Trần Văn Tiến	2003-11-17T17:0...	03634343552	tientran@gmail.c...
staff_thuyquynh	Binh Thị Thủy Q...	2003-11-17T17:0...	036343644552	thuyquynh@gmail...

Hình 3.21 Quản lý thủ thư

3.4.1.2 Code xử lý

Ví dụ sau về cách thử đọc dữ liệu lên từ mongoDB, load dữ liệu các thủ thư lên datagridview, sự kiện buttonLogin_Click sẽ gọi đến hàm LoadInformation trong lớp staff thuộc tầng Business Logic Layer (BLL).

```
public bool LoadDataToGrid()
{
    var info = staff.LoadInformation();

    if (info != null && info.Count > 0)
    {
        DataTable table = new DataTable();
        foreach (var key in info[0].Names)
        {
            table.Columns.Add(key);
        }

        foreach (var doc in info)
        {
            var row = table.NewRow();
            foreach (var key in doc.Names)
            {
                row[key] = doc[key].ToString();
            }
            table.Rows.Add(row);
        }
        dgvStaffs.DataSource = table;
        return true;
    }
    else
    {
        return false;
    }
}
```

```
{
    return false;
}
```

Hàm LoadInformation trong lớp staff sẽ thực hiện việc truy xuất dữ liệu từ MongoDB thông qua hàm Find trong lớp Data Access Layer (DAL). Dưới đây là nội dung hàm CheckLogin

```
public List<BsonDocument> LoadInformation()
{
    string filter = "{}"; // Lấy tất cả document
    string projection = "{ \"_id\": 0 }";
    return dal.Find("staff", filter, projection);
}
```

Hàm List ở lớp DAL có nhiệm vụ kết nối đến MongoDB và truy xuất dữ liệu theo điều kiện lọc được truyền vào. Nội dung cụ thể của hàm Find như sau

```
public List<BsonDocument> Find(string collectionName, string filterString, string projectionString)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    var filter = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(filterString);
    var projection = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(projectionString);
    return collection.Find(filter).Project<BsonDocument>(projection).ToList();
}
```

3.4.5 Quản lý người đọc

Quản lý độc giả (Manage Reader): Hỗ trợ cập nhật thông tin người dùng thư viện (sinh viên, giảng viên...), khóa/mở tài khoản, đặt lại mật khẩu hoặc theo dõi lịch sử mượn sách của từng người dùng.

3.4.5.1 Giao diện

username	readerName	birthdate	phone	email
21110276	Cao Thị Ngọc Ph...	2003-11-17T17:0...	03631810362	21110276@stud...
21110319	Trần Văn Tiến	2003-11-17T17:0...	0315599262	21110319@stud...
21110001	Đinh Thị Thủy G...	2003-11-16T17:0...	03631810362	21110001@stud...
21110002	Nguyễn Văn Hào	2003-12-16T17:0...	0315599262	21110002@stud...
21110003	Nguyễn Thị Thủy...	2002-08-10T00:0...	0909112233	21110003@stud...
21110004	Nguyễn Văn A	2001-04-21T00:0...	087854321	21110004@stud...
21110005	Lê Thị B	2002-05-01T00:0...	0833111222	21110005@stud...
21110006	Phạm Quốc Bảo	2001-07-12T00:0...	090888777	21110006@stud...
21110007	Nguyễn Minh Tuấn	2003-01-19T00:0...	0909000111	21110007@stud...
21110008	Tiến Thị Hồng	2002-02-02T00:0...	090955444	21110008@stud...
21110009	Phạm Duy Khanh	2001-10-15T00:0...	093345666	21110009@stud...
21110010	Nguyễn Thị Hân...	2003-03-08T00:0...	093233444	21110010@stud...
21110011	Bảng Lê Nguyễn...	2003-12-20T00:0...	0909222111	21110011@stud...
21110012	Tần Thị Kim Ngân	2002-09-01T00:0...	0912345678	21110012@stud...
21110013	Vũ Văn Phong	2003-05-16T00:0...	0909011222	21110013@stud...
21110014	Lê Minh Trí	2001-05-05T00:0...	090998776	21110014@stud...
21110015	Ngô Thị Thu Trang	2002-12-12T00:0...	083809000	21110015@stud...

Hình 3.22 Quản lý người đọc

3.4.5.2 Code xử lý

Đoạn mã dưới đây minh họa cách xử lý khi người dùng nhấn nút Tạo trên giao diện. Hàm `buttonCreate_Click` có nhiệm vụ thu thập thông tin từ các ô nhập liệu, sau đó gọi hàm `InsertReader` từ lớp reader thuộc tầng Business Logic Layer (BLL) để lưu dữ liệu vào MongoDB.

```
private void buttonCreate_Click(object sender, EventArgs e)
{
    string userName = tbID.Text;
    string readerName = tbReaderName.Text;
    DateTime birthDate = pickerBirthDate.Value.Date;
    string phone = tbPhone.Text;
    string email = tbEmail.Text;
    string address = tbAddress.Text;
    string position = tbPosition.Text;

    string result = reader.InsertReader(userName,
        readerName,
        birthDate,
        phone,
        email,
        address,
        position);
    if (result == "")
    {
        MessageBox.Show($"Thêm người đọc thành công");
        LoadDataToGrid();
    }
    else
    {
        MessageBox.Show($"Lỗi không thể thêm người đọc\n{result}",
            "Lỗi",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

Hàm `InsertReader` trong lớp BLL sẽ chuẩn bị dữ liệu dưới dạng `BsonDocument` và gọi phương thức `Insert_String` trong lớp DAL để lưu trữ dữ liệu vào MongoDB.

```
public string InsertReader(string username,
    string readerName,
    DateTime birthdate,
    string phone,
    string email,
    string address,
    string position)
{
    var doc = new BsonDocument
    {
        { "username", username },
        { "readerName", readerName },
        { "birthdate", birthdate },
        { "phone", phone },
        { "email", email },
        { "address", address },
        { "position", position }
    };
    return dal.Insert_String("reader", doc);
}
```

Trong tầng DAL, hàm `Insert_String` đảm nhiệm việc kết nối và thực hiện lệnh chèn dữ liệu vào MongoDB. Nếu quá trình thành công, hàm sẽ trả về chuỗi rỗng; ngược lại, nó sẽ trả về thông báo lỗi.

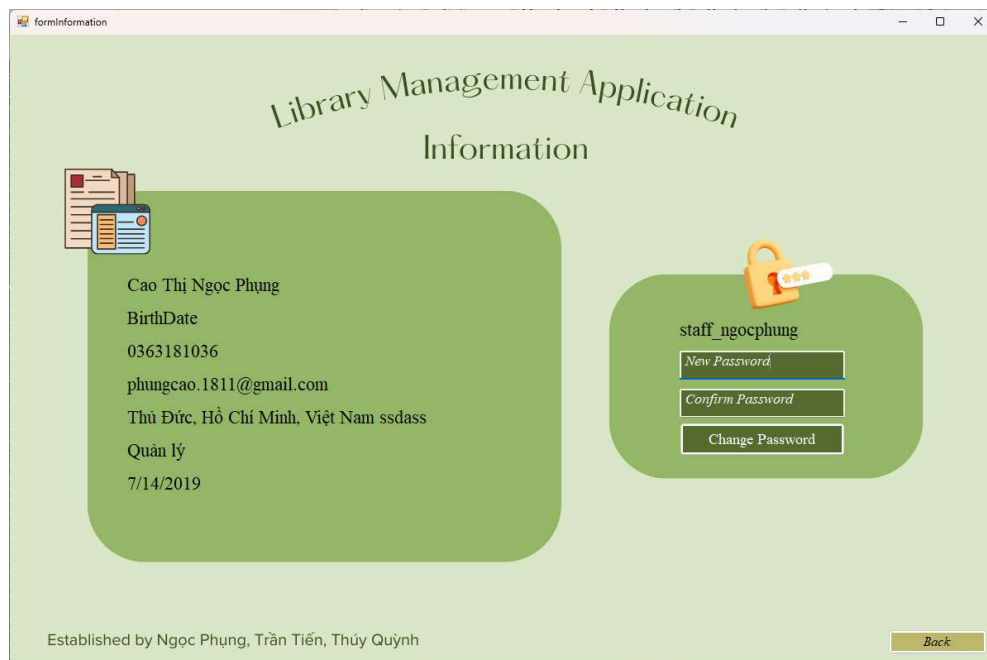
```

public string Insert_String(string collectionName, BsonDocument document)
{
    try
    {
        var collection = db.GetCollection<BsonDocument>(collectionName);
        collection.InsertOne(document);
        return "";
    }
    catch (Exception ex)
    {
        return ex.Message;
    }
}

```

3.4.6 Xem thông tin thủ thư

3.4.6.1 Giao diện



Hình 3.23 Xem thông tin của thủ thư

3.4.6.2 Code xử lý

Khi người dùng nhấn nút **Change Password**, ứng dụng sẽ thực hiện kiểm tra xem mật khẩu mới và mật khẩu xác nhận có trùng khớp hay không. Nếu trùng khớp, hệ thống sẽ gọi hàm UpdateAccount từ lớp account thuộc tầng Business Logic Layer (BLL) để cập nhật mật khẩu mới vào MongoDB.

```

private void buttonChangePassword_Click(object sender, EventArgs e)
{
    if (tbNewPassword.Text == tbConfirmPassword.Text)
    {
        string result = account.UpdateAccount(Session.Username, tbNewPassword.Text);
        if (result == "")
        {
            MessageBox.Show($"Thay đổi mật khẩu thành công");
            LoadDataToGrid();
        }
    }
}

```

```

    }
    else
    {
        MessageBox.Show($"Lỗi không thể thay đổi mật khẩu\n{result}",
            "Lỗi",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
else
{
    MessageBox.Show($"Mật khẩu không khớp, cần kiểm tra lại");
}
}

```

Hàm UpdateAccount trong lớp BLL có nhiệm vụ xây dựng điều kiện lọc (filter) để xác định tài khoản cần cập nhật, đồng thời tạo câu lệnh cập nhật (update) nhằm thay đổi mật khẩu. Sau đó, hàm này gọi đến phương thức Update_String trong lớp DAL để thực hiện thao tác cập nhật trên MongoDB.

```

public string UpdateAccount(string username,
    string password)
{
    var filter = new BsonDocument { { "username", username } };

    var update = Builders<BsonDocument>.Update
        .Set("password", password);
    return dal.Update_String("account", filter, update);
}

```

Trong tầng DAL, phương thức Update_String thực hiện kết nối với MongoDB, truy cập vào collection tương ứng và cập nhật dữ liệu theo filter và nội dung cập nhật được truyền vào.

```

public string Update_String(string collectionName, BsonDocument filterDoc, UpdateDefinition<BsonDocument> updateDef)
{
    try
    {
        var collection = db.GetCollection<BsonDocument>(collectionName);
        collection.UpdateOne(filterDoc, updateDef);
        return "";
    }
    catch (Exception ex)
    {
        return ex.Message;
    }
}

```

3.4.7 Quản lý danh mục

3.4.7.1 Giao diện

categoryId	categoryName	description
1	Sách giáo khoa	Sách dành cho chương trình học chính ...
2	Sách chuyên ng...	Sách phục vụ học tập và nghiên cứu ch...
3	Sách tham khảo	Tài liệu hỗ trợ học tập, nâng cao kiến thức
4	Tài liệu nghiên cứu	Tài liệu dành cho nghiên cứu chuyên sâu
5	Sách kỹ năng mềm	Rèn luyện kỹ năng sống, làm việc
6	Sách tâm lý học	Tâm lý học ứng dụng, phát triển bản thân
7	Sách văn học Vi...	Tác phẩm văn học trong nước
8	Sách văn học nư...	Tác phẩm văn học kinh điển, hiện đại q...
9	Sách khoa học	Tài liệu kiến thức khoa học tự nhiên
10	Sách công nghệ ...	Tài liệu về CNTT, lập trình, AI
11	Sách kinh tế	Kinh doanh, tài chính, quản trị
12	Sách pháp luật	Tài liệu luật pháp và quy định
13	Sách y học	Tài liệu ngành y, chăm sóc sức khỏe
14	Sách ngoại ngữ	Tiếng Anh, Nhật, Hàn, Trung...
15	Từ điển	Các loại từ điển, tra cứu
16	Sách thiếu nhi	Sách cho trẻ em
17	Truyện tranh	Manga, comic, graphic novel
18	Sách lịch sử	Tài liệu và truyện lịch sử
19	Sách du lịch	Hướng dẫn, trải nghiệm du lịch
20	Sách nấu ăn	Ấm thực, công thức món ngon

Hình 3.24 Xem thông tin của thủ thư

3.4.7.1 Code xử lý

Quản lý thể loại (Manage Categories): Cho phép thêm mới, chỉnh sửa, và xóa thông tin các thể loại sách trong hệ thống. Giao diện hỗ trợ người quản trị dễ dàng phân loại sách theo từng chủ đề cụ thể như Khoa học, Văn học, Kỹ thuật,... giúp việc tìm kiếm và thống kê trở nên dễ dàng hơn. Ví dụ dưới đây minh họa cách lấy dữ liệu từ MongoDB và hiển thị danh sách thể loại lên DataGridView. Sự kiện `buttonLoadCategories_Click` sẽ gọi đến hàm `LoadCategories` trong lớp `category` thuộc tầng `Business Logic Layer (BLL)`:

```
public void LoadDataToGrid()
{
    var info = category.LoadInformation();
    info = info.OrderBy(doc => int.Parse(System.Text.RegularExpressions.Regex.Match(doc["categoryId"].ToString(),
@"^(\d+)$").Value)).ToList();
    if (info != null && info.Count > 0)
    {
        DataTable table = new DataTable();
        foreach (var key in info[0].Names)
        {
            table.Columns.Add(key);
        }
        foreach (var doc in info)
        {
            var row = table.NewRow();
            foreach (var key in doc.Names)
            {
                row[key] = doc[key].ToString();
            }
        }
    }
}
```

```

        table.Rows.Add(row);
    }
    dgvCategories.DataSource = table;
}
}

```

Hàm LoadCategories trong lớp category sẽ thực hiện việc truy xuất dữ liệu từ MongoDB bằng cách gọi phương thức Find trong lớp Data Access Layer (DAL). Nội dung cụ thể như sau:

```

public List<BsonDocument> LoadCategories()
{
    string filter = "{}"; // Lấy tất cả document
    string projection = "{ \"_id\": 0 }"; // Bỏ trường _id
    return dal.Find("category", filter, projection);
}

```

Hàm Find trong lớp DAL có nhiệm vụ kết nối đến MongoDB và truy vấn dữ liệu từ collection category dựa trên điều kiện lọc được truyền vào:

```

public List<BsonDocument> Find(string collectionName, string filterString, string projectionString)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    var filter = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(filterString);
    var projection = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(projectionString);
    return collection.Find(filter).Project<BsonDocument>(projection).ToList();
}

```

3.4.8 Quản lý tác giả

3.4.8.1 Giao diện

authorId	authorName	dateOfBirth	nationality	description
1	Yuval Noah Harari	1976-02-24T00:00:00	Israeli	Tác giả
2	Napoleon Hill	1883-10-26T00:00:00	American	Cha đẻ
3	Dale Carnegie	1888-11-24T00:00:00	American	Tác giả
4	Paulo Coelho	1947-08-24T00:00:00	Brazilian	Tác giả
5	Robert T. Kiyosaki	1947-04-08T00:00:00	American	Tác giả
6	Rosie Nguyễn	1987-06-01T00:00:00	Vietnamese	Tác giả
7	Nguyễn Duy Cận	1975-03-10T00:00:00	Vietnamese	Tác giả
8	Daniel Kahneman	1934-03-05T00:00:00	American	Nhà tâm lý học
9	Stephen R. Covey	1932-10-24T00:00:00	American	Tác giả
10	Adam Khoo	1974-04-08T00:00:00	Singaporean	Doanh nhân
11	Ichiro Kihimi	1956-01-01T00:00:00	Japanese	Băng đảng
12	Fumitake Koga	1973-01-01T00:00:00	Japanese	Băng đảng
13	Og Mandino	1923-12-12T00:00:00	American	Tác giả
14	Keyes	1960-01-01T00:00:00	Chinese	Tác giả
15	Blon Musk	1971-06-28T00:00:00	American	Doanh nhân

Hình 3.25 Quản lý tác giả

3.4.8.2 Code xử lý

Quản lý tác giả (Manage Authors): Cho phép thêm, sửa, xóa thông tin các tác giả trong hệ thống thư viện. Quản trị viên có thể tìm kiếm, cập nhật và hiển thị danh sách tác giả trên giao diện người dùng. Ví dụ sau minh họa cách đọc dữ liệu từ MongoDB, tải dữ liệu các tác giả lên DataGridView. Sự kiện buttonLoadAuthors_Click sẽ gọi đến hàm LoadAuthors trong lớp author thuộc tầng Business Logic Layer (BLL):

```
public bool LoadDataToGrid()
{
    var info = author.LoadInformation();
    info = info.OrderBy(doc => Convert.ToInt32(doc["authorId"].ToString())).ToList();
    if (info != null && info.Count > 0)
    {
        DataTable table = new DataTable();
        foreach (var key in info[0].Names)
        {
            table.Columns.Add(key);
        }
        foreach (var doc in info)
        {
            var row = table.NewRow();
            foreach (var key in doc.Names)
            {
                row[key] = doc[key].ToString();
            }
            table.Rows.Add(row);
        }
        dgvAuthors.DataSource = table;
        return true;
    }
    else
    {
        return false;
    }
}
```

Hàm LoadAuthors trong lớp author sẽ thực hiện việc truy xuất dữ liệu từ MongoDB thông qua hàm Find trong lớp Data Access Layer (DAL). Nội dung của hàm LoadAuthors như sau:

```
public List<BsonDocument> LoadAuthors()
{
    string filter = "{}"; // Lấy tất cả document
    string projection = "{ \"_id\": 0 }"; // Không lấy _id
    return dal.Find("author", filter, projection);
}
```

Hàm Find ở lớp DAL có nhiệm vụ kết nối đến MongoDB và truy xuất dữ liệu từ collection author theo điều kiện lọc. Dưới đây là nội dung cụ thể của hàm:

```
public List<BsonDocument> Find(string collectionName, string filterString, string projectionString)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    var filter = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(filterString);
    var projection = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(projectionString);
    return collection.Find(filter).Project<BsonDocument>(projection).ToList();
}
```

3.4.9 Quản lý sách

3.4.9.1 Giao diện

bookId	title	authorName	publishedYear	categoryName
B001	Sapiens: Lịch sử...	Yuval Noah Harari	2011-01-01T00:0...	Sách lịch sử
B002	Homo Deus: Lối...	Yuval Noah Harari	2015-01-01T00:0...	Sách khoa học
B003	21 bài học cho th...	Yuval Noah Harari	2018-01-01T00:0...	Sách khoa học
B004	Think and Grow ...	Napoleon Hill	1937-01-01T00:0...	Sách văn học nư
B005	Đắc nhân tâm	Dale Carnegie	1936-01-01T00:0...	Sách xã hội học
B006	Nhà giả kim	Paulo Coelho	1988-01-01T00:0...	Văn học nước ng
B007	Chưa giàu chưa ng...	Robert T. Kiyosaki	1997-01-01T00:0...	Sách tham khảo
B008	Tuổi trẻ đáng giá...	Rosie Nguyễn	2016-01-01T00:0...	Sách văn học Vi
B009	Một đời quản trị	Nguyễn Duy Cuô...	2021-01-01T00:0...	Sách kinh tế
B010	Tư duy nhanh và...	Daniel Kahneman	2011-01-01T00:0...	Sách tâm lý học
B011	7 thói quen của n...	Stephen R. Covey	1989-01-01T00:0...	Sách tham khảo
B012	Tôi tài giỏi, bạn c...	Adam Khoo	2002-01-01T00:0...	Sách kỹ năng mề
B013	Đảm bị ghét	Ichiro Kishimi & F...	2013-01-01T00:0...	Sách tâm lý học
B014	Bí mật của Phan...	Og Mandino	1968-01-01T00:0...	Sách chuyển ng
B015	Thiền tái bản trấ...	Keyes	2016-01-01T00:0...	Sách tâm lý học

Hình 3.26 Quản lý book

3.4.9.2 Code xử lý

Quản lý sách (Manage Books): Cho phép thêm mới, chỉnh sửa, xóa thông tin các đầu sách trong thư viện. Quản trị viên có thể theo dõi danh mục sách, cập nhật số lượng, thể loại, tình trạng sách và các thông tin liên quan khác. Ví dụ sau minh họa cách truy xuất dữ liệu từ MongoDB và hiển thị danh sách sách lên DataGridView. Sự kiện buttonLoadBooks_Click sẽ gọi đến hàm LoadBooks trong lớp book thuộc tầng Business Logic Layer (BLL):

```
private void LoadBooksToGrid()
{
    var list = book.LoadInformation();
}
```

```

        list = list.OrderBy(doc => int.Parse(System.Text.RegularExpressions.Regex.Match(doc["bookId"].ToString(),
@"^\d+").Value)).ToList();
        if (list != null && list.Count > 0)
        {
            DataTable table = new DataTable();
            foreach (var key in list[0].Names)
            {
                table.Columns.Add(key);
            }
            foreach (var doc in list)
            {
                var row = table.NewRow();
                foreach (var key in doc.Names)
                {
                    row[key] = doc[key].ToString();
                }
                table.Rows.Add(row);
            }
            dgvBooks.DataSource = table;
        }
    }
}

```

Hàm LoadBooks trong lớp book sẽ gọi đến phương thức Find trong lớp Data Access Layer (DAL) để truy vấn dữ liệu từ MongoDB. Nội dung hàm LoadBooks như sau:

```

public List<BsonDocument> LoadBooks()
{
    string filter = "{}"; // Lấy tất cả sách
    string projection = "{ \"_id\": 0 }"; // Không lấy trường _id
    return dal.Find("book", filter, projection);
}

```

Hàm Find tại lớp DAL đảm nhiệm việc kết nối MongoDB và thực hiện truy vấn dữ liệu theo filter và projection được truyền vào:

```

public List<BsonDocument> Find(string collectionName, string filterString, string projectionString)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    var filter = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(filterString);
    var projection = MongoDB.Bson.Serialization.BsonSerializer.Deserialize<BsonDocument>(projectionString);
    return collection.Find(filter).Project<BsonDocument>(projection).ToList();
}

```

3.4.10 Quản lý mượn sách

3.4.10.1 Giao diện

Loan ID	Reader	Book	Borrow Date	Due Date
L001	21110001	B001	4/1/2025	4/1/25
L002	21110002	B002	4/10/2025	4/2/25
L003	21110003	B003	4/15/2025	4/3/25
L004	21110004	B004	3/20/2025	4/4/25
L005	21110005	B005	4/5/2025	4/2/25
L006	21110006	B006	3/30/2025	4/1/25
L007	21110007	B007	4/2/2025	4/1/25
L008	21110008	B008	4/12/2025	4/2/25
L009	21110009	B009	3/25/2025	4/3/25
L010	21110010	B010	4/18/2025	5/3/25
L011	21110011	B011	3/1/2025	3/1/25
L012	21110012	B012	4/3/2025	4/1/25
L013	21110013	B013	4/10/2025	4/2/25
L014	21110014	B014	2/25/2025	3/1/25
L015	21110015	B015	4/15/2025	4/3/25

Hình 3.27 Quản lý mượn sách

3.4.10.1 Code xử lý

Logic xử lý load dữ liệu mượn sách và giao dịch:

- Khi người dùng khởi động form (UI), phương thức Load sẽ gọi service tương ứng (LoanService.GetAll() hoặc TransactionService.GetAll()), nhận về danh sách DTO.
- BLL (GetAll) thực hiện truy vấn qua DAL (hàm Find), ánh xạ BsonDocument sang DTO, xử lý format và timezone.
- UI gán DTO vào BindingSource và hiển thị qua DataGridView.

Hàm Load và hiển thị danh sách Loan

```
private void formLoans_Load(object sender, EventArgs e)
{
    // 1) UI gọi BLL
    var list = _loanService.GetAll();

    // 2) UI dùng DataGridView
    _bs = new BindingSource { DataSource = list };
    dgvLoans.AutoGenerateColumns = false;
    dgvLoans.DataSource = _bs;
}
```

BLL (LoanService.GetAll): Truy vấn DAL và ánh xạ DTO

```
public List<LoanDto> GetAll()
{
```

```

var docs = _dal.Find(
    "loan", "{}",
    "{ loanId:1, readerUsername:1, bookId:1, borrowDate:1, dueDate:1, returnDate:1, status:1 }"
);
return docs.Select(d => new LoanDto {
    LoanId = d["loanId"].AsString,
    ReaderUsername = d["readerUsername"].AsString,
    BookId = d["bookId"].AsString,
    BorrowDate = d["borrowDate"].ToLocalTime(),
    DueDate = d["dueDate"].ToLocalTime(),
    ReturnDate = d.Contains("returnDate") ? (DateTime?)d["returnDate"].ToLocalTime() : null,
    Status = d["status"].AsString
}).ToList();
}

```

DAL (Find): Truy vấn MongoDB

```

public List<BsonDocument> Find(string collectionName, string filterJson, string projectionJson)
{
    var col = db.GetCollection<BsonDocument>(collectionName);
    var filter = BsonSerializer.Deserialize<BsonDocument>(filterJson);
    var find = col.Find(filter);
    if (!string.IsNullOrEmpty(projectionJson))
        find = find.Project<BsonDocument>(BsonSerializer.Deserialize<BsonDocument>(projectionJson));
    return find.ToList();
}

```

3.4.11 Quản lý giao dịch

3.4.11.1 Giao diện

Loan ID	Reader Username	Book ID	Staff Username	Date
L001	21110001	B001	staff_ngocphung	5/1/2025 4:00
L001	21110001	B001	staff_ngocphung	5/3/2025 9:30
L002	21110002	B002	staff_tien	5/5/2025 6:15
L002	21110002	B002	admins	5/16/2025 6:12
L003	21110003	B003	test	5/17/2025 9:24
L003	21110003	B003	test	5/17/2025 9:24
L003	21110003	B003	test	5/17/2025 9:23
L003	21110003	B003	test	5/17/2025 9:42
L005	21110005	B005	staff_ngocphung	5/8/2025 4:35
*				

Hình 3.28 Quản lý mượn sách

3.4.11.2 Code xử lý

Logic xử lý tạo Transaction:

- UI thu thập dữ liệu từ các control, rồi gọi BLL (Create).

- BLL validate input, chuyển DateTime sang UTC, xây dựng BsonDocument, và gọi DAL (Insert).
- DAL tương tác trực tiếp với MongoDB, trả về kết quả (thành công/ thất bại).
- BLL trả kết quả về UI, UI reload lại dữ liệu qua GetAll() và cập nhật giao diện.

Hàm tạo Transaction

```
private void btnSave_Click(object sender, EventArgs e) {
    // 1) Parse amount
    if (!decimal.TryParse(txtAmount.Text, out var amount)) { MessageBox.Show("Số tiền không hợp lệ!");
    txtAmount.Focus();
    return; }
    // 2) Lấy dữ liệu từ form
    var loanId = cboLoan.SelectedValue.ToString();
    var type = cboType.Text;
    var date = dtpDate.Value;
    var staffUsername = Session.Username; // nhớ using Business_Logic_Layer

    // 3) Gọi service lưu
    var ok = _trsService.Create(loanId, staffUsername, date, type, amount);
    if (!ok)
    {
        MessageBox.Show("Lỗi khi lưu giao dịch!");
        return;
    }

    MessageBox.Show("Đã lưu giao dịch!");

    // 4) Reload toàn bộ danh sách & filter lại
    _allTransactions = _trsService.GetAll();
    RefreshGrid();

    // 5) Về chế độ view-only
    ToggleEditMode(false);
}
```

BLL (TransactionService.Create) Gọi DAL tương ứng

```
public bool Create(string loanId, string staffUsername,
    DateTime txnDate, string type, decimal amount)
{
    if (amount < 0) throw new ArgumentException();
    var doc = new BsonDocument {
        { "loanId", loanId },
        { "staffUsername", staffUsername },
        { "transactionDate", txnDate.ToUniversalTime() },
        { "type", type },
        { "amount", amount }
    };
    return _dal.Insert("transaction", doc);
}
```

DAL: Insert Truy vấn MongoDB

```
public bool Insert(string collectionName, BsonDocument document)
{
    try
    {
        var collection = db.GetCollection<BsonDocument>(collectionName);
        collection.InsertOne(document);
        return true;
    }
    catch
    {
        return false;
    }
}
```

3.4.12 Thống kê số liệu của thư viện

3.4.12.1 Giao diện



Hình 3.29 Thông kê các số liệu

3.4.12.2 Code xử lý

Chức năng thống kê tổng số tiền thanh toán theo từng năm được thực hiện khi người dùng nhấn nút Revenue trên giao diện. Lúc này, hàm `buttonRevenue_Click` sẽ được kích hoạt và gọi đến phương thức `GetTotalPaymentPerYear` từ lớp `transaction` thuộc tầng Business Logic Layer (BLL) để lấy dữ liệu thống kê.

```
private void buttonRevenue_Click(object sender, EventArgs e)
{
    var result = transaction.GetTotalPaymentPerYear();

    DataTable dt = new DataTable();
    dt.Columns.Add("Năm");
    dt.Columns.Add("Tổng thanh toán");
    foreach (var doc in result)
    {
        int year = doc["_id"].ToInt32();
        double total = doc["totalPaid"].ToDouble();
        dt.Rows.Add(year, total);
    }
    dgvStatistic.DataSource = dt;
}
```

Hàm `GetTotalPaymentPerYear` trong lớp `transaction` sẽ khởi tạo một pipeline (chuỗi các bước xử lý) dùng trong truy vấn aggregate của MongoDB. Pipeline này bao gồm ba bước:

Lọc dữ liệu (`$match`): chỉ lấy các giao dịch có type là "Payment".

Nhóm dữ liệu (\$group): tính tổng số tiền (amount) theo từng năm của trường transactionDate.

Sắp xếp (\$sort): sắp xếp kết quả theo thứ tự tăng dần của năm.

Sau khi tạo pipeline, hàm sẽ gọi đến phương thức Aggregate ở lớp DAL để thực hiện truy vấn dữ liệu trực tiếp trên MongoDB.

```
public List<BsonDocument> GetTotalPaymentPerYear()
{
    var pipeline = new[]
    {
        new BsonDocument("$match", new BsonDocument("type", "Payment")),
        new BsonDocument("$group", new BsonDocument
        {
            { "_id", new BsonDocument("$year", "$transactionDate") },
            { "totalPaid", new BsonDocument("$sum", "$amount") }
        }
    ),
        new BsonDocument("$sort", new BsonDocument("_id", 1))
    };
    return _dal.Aggregate("transaction", pipeline);
}
```

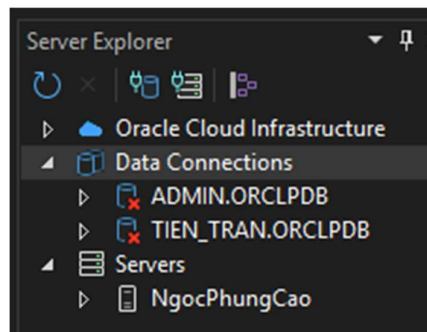
Phương thức Aggregate trong lớp DAL sẽ nhận vào tên collection và pipeline, sau đó kết nối với MongoDB để thực hiện truy vấn tổng hợp.

```
public List<BsonDocument> Aggregate(string collectionName, PipelineDefinition<BsonDocument, BsonDocument> pipeline)
{
    var collection = db.GetCollection<BsonDocument>(collectionName);
    return collection.Aggregate(pipeline).ToList();
}
```

3.5 Một vài so sánh giữa MongoDB và SQL Server

3.5.1 Kết nối

SQL Server có thể kết nối trực tiếp với Server Explorer



Hình 3.30 Kết nối với SQL server

MongoDB phải tải driver và kết nối thông qua chuỗi string

```
private string connectionString = "mongodb+srv://hcmutelibrary:library@clusterngocphung.i6mtwhx.mongodb.net/?retryWrites=true&w=majority&appname=hcmutelibrary";
private IMongoDatabase db;
public string name;

6 references
public DAL()
{
    var client = new MongoClient(connectionString);
    db = client.GetDatabase("LibraryDB");
}
```

Hình 3.30 kết nối với MongoDB

3.5.2 Insert dữ liệu vào cơ sở dữ liệu

Code insert dữ liệu được truyền từ .Net đến SQL Server

```
public bool InsertBooks(int ID,
    string Title,
    string Author,
    int Year,
    string Genre,
    int Quantity)
{
    SqlCommand command = new SqlCommand("INSERT INTO Books " +
        "VALUES (@id,@title,@author,@year,@genre,@quantity) ", mydatabase.getConnection());
    command.Parameters.Add("@id", SqlDbType.Int).Value = ID;
    command.Parameters.Add("@title", SqlDbType.NVarChar).Value = Title;
    command.Parameters.Add("@author", SqlDbType.NVarChar).Value = Author;
    command.Parameters.Add("@year", SqlDbType.Int).Value = Year;
    command.Parameters.Add("@genre", SqlDbType.NVarChar).Value = Genre;
    command.Parameters.Add("@quantity", SqlDbType.Int).Value = Quantity;
    mydatabase.openConnection();

    if ((command.ExecuteNonQuery()) == 1)
    {
        mydatabase.closeConnection();
        return true;
    }
    else
    {
        mydatabase.closeConnection();
        return false;
    }
}
```

Code insert dữ liệu được truyền từ MongoDB đến SQL Server

```
public bool InsertBook(
    string bookId,
    string title,
    string authorName,
    DateTime publishedYear,
    string category)
{
    var doc = new BsonDocument
    {
        { "bookId", bookId },
        { "title", title },
        { "authorName", authorName },
        { "publishedYear", publishedYear },
        { "categoryName", category }
    };
    return dal.Insert("book", doc);
}
```

```
5 references
public bool Insert(string collectionName, BsonDocument document)
{
    try
    {
        var collection = db.GetCollection<BsonDocument>(collectionName);
        collection.InsertOne(document);
        return true;
    }
    catch
    {
        return false;
    }
}
```

3.5.3 Update dữ liệu vào cơ sở dữ liệu

Code update dữ liệu được truyền từ .Net đến SQL Server

```
1 reference
public bool UpdateBooks(int ID,
    string Title,
    string Author,
    int Year,
    string Genre,
    int Quantity)
{
    SqlCommand command = new SqlCommand("UPDATE Books " +
        "SET Title = @title, " +
        "Author = @author, " +
        "PublicationYear = @year, " +
        "Genre = @genre, " +
        "Quantity = @quantity " +
        "WHERE BookID = @id ", mydatabase.getConnection());
    command.Parameters.Add("@id", SqlDbType.Int).Value = ID;
    command.Parameters.Add("@title", SqlDbType.NVarChar).Value = Title;
    command.Parameters.Add("@author", SqlDbType.NVarChar).Value = Author;
    command.Parameters.Add("@year", SqlDbType.Int).Value = Year;
    command.Parameters.Add("@genre", SqlDbType.NVarChar).Value = Genre;
    command.Parameters.Add("@quantity", SqlDbType.Int).Value = Quantity;
    mydatabase.openConnection();

    if ((command.ExecuteNonQuery()) == 1)
    {
        mydatabase.closeConnection();
        return true;
    }
    else
    {
        mydatabase.closeConnection();
        return false;
    }
}
```

Code update dữ liệu được truyền từ .Net đến MongoDB

```
1 reference
public bool UpdateBook(
    string bookId,
    string title,
    string authorName,
    DateTime publishedYear,
    string category)
{
    var filter = new BsonDocument { { "bookId", bookId } };

    var update = Builders<BsonDocument>.Update
        .Set("title", title)
        .Set("authorName", authorName)
        .Set("publishedYear", publishedYear)
        .Set("categoryName", category);
    return dal.Update("book", filter, update);
}
```

3.5.4 Delete dữ liệu vào cơ sở dữ liệu

Code delete dữ liệu được truyền từ .Net đến SQL Server

```
1 reference
public bool DeleteBooks(int ID)
{
    SqlCommand command = new SqlCommand("DELETE FROM Books WHERE BookID = @id", mydatabase.getConnection());
    command.Parameters.Add("@id", SqlDbType.Int).Value = ID;
    mydatabase.openConnection();

    if ((command.ExecuteNonQuery()) == 1)
    {
        mydatabase.closeConnection();
        return true;
    }
    else
    {
        mydatabase.closeConnection();
        return false;
    }
}
```

Code update dữ liệu được truyền từ .Net đến MongoDB

```
1 reference
public bool DeleteBook(string bookId)
{
    var filter = new BsonDocument { { "bookId", bookId } };
    return dal.Delete("book", filter);
}
```

PHẦN KẾT LUẬN

Trong bối cảnh chuyển đổi số mạnh mẽ trong lĩnh vực giáo dục, việc hiện đại hóa các hệ thống quản lý truyền thống, đặc biệt là thư viện, trở nên hết sức cần thiết. Đồ án “Xây dựng hệ thống quản lý thư viện sử dụng MongoDB và .NET” được triển khai nhằm giải quyết những hạn chế đang tồn tại trong hệ thống cũ như giao diện lỗi thời, khả năng mở rộng kém và thiếu tính linh hoạt trong xử lý dữ liệu.

Thông qua quá trình phân tích, thiết kế và xây dựng hệ thống, nhóm đã áp dụng mô hình cơ sở dữ liệu NoSQL (MongoDB) để tối ưu hóa khả năng lưu trữ và truy xuất thông tin phi cấu trúc, đồng thời sử dụng .NET làm nền tảng phát triển ứng dụng nhằm đảm bảo tính ổn định, dễ triển khai và dễ mở rộng. Hệ thống mới cho phép quản lý đầy đủ các thực thể như sách, độc giả, tác giả, giao dịch, nhân viên... với giao diện trực quan, phân quyền rõ ràng và hiệu năng cải thiện đáng kể.

Kết quả triển khai thử nghiệm cho thấy hệ thống hoạt động ổn định, dễ sử dụng và phù hợp với nhu cầu thực tế tại các thư viện trường học hoặc tổ chức quy mô vừa và nhỏ. Dù vẫn còn những hạn chế nhất định về tính năng nâng cao hoặc hỗ trợ đa nền tảng, đồ án đã đặt nền móng vững chắc cho việc phát triển và hoàn thiện hệ thống quản lý thư viện hiện đại trong tương lai.

Qua đồ án này, nhóm không chỉ củng cố kiến thức chuyên môn về lập trình, cơ sở dữ liệu và thiết kế hệ thống, mà còn rèn luyện kỹ năng làm việc nhóm, phân tích yêu cầu thực tế và giải quyết vấn đề theo hướng ứng dụng. Đây là bước khởi đầu quan trọng để phát triển các dự án phần mềm quy mô lớn và phục vụ tốt hơn cho nhu cầu xã hội.

TÀI LIỆU THAM KHẢO

- MongoDB Documentation (2024).
- Stonebraker, M. (2007). *One Size Fits All: An Idea Whose Time Has Come and Gone*. Communications of the ACM.
- Microsoft Docs (2023). *Windows Forms Overview*.
- Troelsen, A. (2019). *Pro C# 8 with .NET Core*. Apress.